

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

PRÁCTICA EXPERIMENTAL – UNIDAD IV

Validación, Gestión de Requisitos y Herramientas CASE

ASIGNATURA:

INGENIERÍA DE REQUERIMIENTOS

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN

INTEGRANTES:

CEDEÑO AVILA WINSTON DAMIAN, CI: 0942833492, wcedenoa2@uteq.edu.ec

CORDOVA CARREÑO MAYRA LUCILA, CI: 0750286775, mcordovac2@uteq.edu.ec

MENDOZA PÁRRAGA ANDY JOHEL, CI: 1251401590, amendozap9@uteq.edu.ec, Titular PFC

EQUIPO:

B

CURSO/PARALELO:

CUARTO SEMESTRE PARALELO “B”

SISTEMA PFC:

MundiPets (Andy Mendoza – Titular PFC)

REPOSITORIO GITHUB:

<https://github.com/andymendozap03/ISR401-PFC-ERS-EQUIPOB.git>

QUEVEDO – LOS RÍOS – ECUADOR
2026 – 2027 PPA

Índice

1.	Resumen y objetivos	3
1.1.	Objetivo general	3
1.2.	Objetivos específicos	3
2.	Fundamento teórico	4
2.1.	Marco normativo	4
2.1.1.	¿Qué regula la validación y gestión de requisitos?	4
2.1.2.	IEEE/ISO/IEC 29148:2018 – Proceso de validación de requisitos de software	4
2.1.3.	IEEE/ISO/IEC 15288:2023 y 12207:2017 – Procesos de control de cambios	4
2.1.4.	ISO/IEC 25010:2023 – Modelo de calidad aplicado al ERS	5
2.1.5.	SWEBOK v4.0 y Pohl (2025) Parte V	5
2.1.6.	El CCB según PMBOK 7. ^a edición	5
2.2.	Validación de requisitos	5
2.2.1.	Verificación vs. validación: diferencias conceptuales	6
2.2.2.	Principios de validación de requisitos	6
2.2.3.	Técnicas de validación	6
2.2.4.	Inspección Fagan: origen, fases y roles	6
2.2.5.	Métricas de inspección	7
2.2.6.	Caso de estudio o ejemplo aplicado	7
2.3.	Gestión de requisitos	7
2.3.1.	Gestión de la configuración del ERS	8
2.3.2.	Línea base (baseline) y control de versiones	8
2.3.3.	Atributos de los requisitos	8
2.3.4.	Trazabilidad: pre-traceability y post-traceability	8
2.3.5.	Matriz de trazabilidad	9
2.3.6.	Proceso de cambio: RFC, análisis de impacto, CCB, implementación y cierre	9
2.3.7.	Métricas de volatilidad de requisitos	9
2.4.	Herramientas CASE para IR	10
2.4.1.	Clasificación de herramientas CASE	10
2.4.2.	Funcionalidades esperables en gestión de requisitos	10
2.4.3.	Criterios de selección de una herramienta CASE	11
2.4.4.	Limitaciones y costos	11
2.5.	IR en procesos ágiles	11
2.5.1.	Product backlog, historias de usuario y criterios de aceptación	11
2.5.2.	Grooming y Definition of Ready / Definition of Done	11
2.5.3.	BDD y formato Gherkin	12
2.5.4.	Trazabilidad en entornos ágiles	12
2.5.5.	IR documental frente a IR ágil: ¿cuándo conviene cada enfoque?	12
3.	Inspección formal del ERS (método Fagan)	13
3.1.	Planificación y roles	13
3.2.	Preparación individual	13
3.3.	Acta de la reunión de inspección	13
3.4.	Métricas de inspección	14
4.	Correcciones aplicadas	15
4.1.	Corrección de defectos críticos y mayores	15
4.2.	Defectos menores	16
5.	Gestión del cambio y Change Control Board (CCB)	17
5.1.	RFC-01 – Cambio de alcance (nuevo requisito)	17
5.2.	RFC-02 – Cambio de calidad (modificación de un RNF)	17
5.3.	RFC-03 – Cambio de restricción o normativa	18
5.4.	Acta del CCB	18
5.5.	Versión resultante del ERS	19

6.	Trazabilidad en herramienta CASE	19
6.1.	Estructura del tablero	19
6.2.	Campos personalizados	19
6.3.	Matriz de trazabilidad	19
6.4.	Evidencias del tablero	20
7.	Línea base del ERS con Git	23
7.1.	Repositorio y estructura	23
7.2.	Commits semánticos	23
7.3.	Tag anotado de línea base	24
7.4.	CHANGELOG.md	24
7.5.	Historial de commits (evidencia)	24
8.	Comparativa de herramientas CASE	26
9.	IR tradicional frente a IR ágil	28
10.	Conclusiones críticas	30
11.	Distribución del trabajo	31
12.	Referencias	32
	Anexo A — Lista de verificación de inspección (checklists individuales)	34
	Anexo B — Registro de defectos	35
	Anexo C — Formularios RFC y acta del CCB	36
	Anexo D — Exportación CSV del backlog	37
	Anexo E — Capturas del tablero CASE y del repositorio Git	38
	Anexo F — Referencias IEEE y declaración de uso de Inteligencia Artificial	39

1. Resumen y objetivos

1.1. Objetivo general

Aplicar técnicas formales de validación de requisitos, gestionar cambios mediante un Change Control Board (CCB) simulado, implementar la trazabilidad del ERS en una herramienta CASE, establecer una línea base con Git y comparar metodológicamente la Ingeniería de Requisitos (IR) tradicional frente a la IR ágil, sobre el ERS real del Proyecto Fin de Curso MundiPets.

1.2. Objetivos específicos

1. Ejecutar una inspección formal tipo Fagan sobre el ERS del PFC, con roles diferenciados, lista de verificación y registro de defectos.
2. Calcular e interpretar métricas de inspección (densidad de defectos, distribución por tipo y severidad, tasa de corrección).
3. Tramitar tres solicitudes formales de cambio (RFC) con análisis de impacto sustentado en la matriz de trazabilidad.
4. Deliberar y decidir en un CCB simulado, dejando acta motivada y versión actualizada del ERS.
5. Construir la trazabilidad bidireccional del ERS en una herramienta CASE de gestión (Jira o Trello) con campos personalizados y enlaces verificables.
6. Establecer y publicar la línea base del ERS con control de versiones (commit semántico, tag anotado y `CHANGELOG.md`).
7. Comparar herramientas CASE de IR y contrastar la IR tradicional frente a la IR ágil, cerrando con conclusiones críticas propias.

2. Fundamento teórico

2.1. Marco normativo

La ingeniería de requisitos se apoya en estándares, cuerpos de conocimiento y marcos metodológicos que establecen principios para especificar, validar, mantener y controlar los requisitos durante el ciclo de vida de un sistema. Entre los principales referentes se encuentran ISO/IEC/IEEE 29148:2018 para los procesos de ingeniería de requisitos, ISO/IEC/IEEE 15288:2023 e ISO/IEC/IEEE 12207:2017 para los procesos del ciclo de vida, ISO/IEC 25010:2023 para las características de calidad, así como SWEBOK v4.0 y los fundamentos contemporáneos de ingeniería de requisitos desarrollados por Pohl. Estos referentes permiten abordar la calidad de los requisitos no únicamente desde su redacción, sino también desde su verificabilidad, trazabilidad y evolución durante el desarrollo.

La relevancia de estos mecanismos continúa siendo respaldada por investigaciones recientes. Un estudio de mapeo publicado en *Requirements Engineering* señala que la calidad de los requisitos constituye un área multidimensional cuya evaluación involucra diferentes propiedades, defectos y mecanismos de aseguramiento [1]. Esta perspectiva justifica la utilización conjunta de estándares y prácticas de ingeniería para establecer criterios consistentes de calidad y control de los requisitos.

2.1.1. ¿Qué regula la validación y gestión de requisitos?

La validación y la gestión de requisitos comprenden actividades destinadas a asegurar que los requisitos representen adecuadamente las necesidades de los interesados y permanezcan controlados durante la evolución del proyecto. La validación se orienta a determinar si los requisitos especificados corresponden con las necesidades y objetivos que el sistema debe satisfacer, mientras que la gestión considera aspectos como los atributos de los requisitos, su trazabilidad, priorización, versionado y tratamiento de cambios.

Estas actividades no deben entenderse como procesos aislados. La modificación de un requisito puede afectar otros requisitos y artefactos asociados, por lo que resulta necesario mantener relaciones que permitan conocer su procedencia y las consecuencias de su evolución. Investigaciones recientes sobre ingeniería de requisitos identifican precisamente la validación y la gestión como actividades fundamentales: la primera busca comprobar la calidad y adecuación de la información obtenida, mientras que la segunda permite seguir los cambios producidos en los requisitos y mantenerlos acordes con las necesidades de los stakeholders [2].

2.1.2. IEEE/ISO/IEC 29148:2018 – Proceso de validación de requisitos de software

ISO/IEC/IEEE 29148:2018 constituye uno de los principales referentes normativos para la ingeniería de requisitos de sistemas y software. El estándar establece procesos y disposiciones relacionadas con la identificación, formulación, análisis y gestión de requisitos a lo largo del ciclo de vida [3]. Dentro de este contexto, la validación permite determinar si los requisitos y la especificación resultante representan adecuadamente las necesidades de los interesados y los objetivos establecidos para el sistema.

La calidad de un requisito implica que pueda comprenderse y utilizarse posteriormente para actividades de diseño, implementación y comprobación. Por esta razón, la especificación debe procurar características que reduzcan ambigüedades, inconsistencias y dificultades de interpretación. La investigación empírica reciente confirma que la calidad de requisitos continúa siendo un problema relevante en ingeniería de software y que sus atributos y defectos requieren métodos sistemáticos de evaluación [1].

2.1.3. IEEE/ISO/IEC 15288:2023 y 12207:2017 – Procesos de control de cambios

ISO/IEC/IEEE 15288:2023 proporciona un marco de procesos para el ciclo de vida de sistemas, mientras que ISO/IEC/IEEE 12207:2017 establece procesos aplicables al ciclo de vida del software [4, 5]. Dentro de este marco, los requisitos no se consideran elementos estáticos: pueden evolucionar como consecuencia de nuevas necesidades, restricciones, decisiones técnicas o cambios en el contexto del sistema.

Por ello, el control de cambios debe permitir identificar una modificación, analizar sus posibles consecuencias y mantener consistentes los elementos afectados. En ingeniería de requisitos esto adquiere especial importancia porque una modificación puede repercutir en requisitos relacionados, diseño, implementación y pruebas. Consecuentemente, la trazabilidad constituye un mecanismo de apoyo fundamental para determinar dichas relaciones.

Esta necesidad está respaldada por estudios contemporáneos sobre trazabilidad, que señalan que mantener los requisitos actualizados y comprender su contexto resulta especialmente importante conforme aumenta el tamaño y complejidad del proyecto. La trazabilidad permite recuperar información sobre el origen, las modificaciones y las personas involucradas en un requisito, proporcionando la información necesaria para gestionar su evolución [6].

2.1.4. ISO/IEC 25010:2023 – Modelo de calidad aplicado al ERS

ISO/IEC 25010:2023 establece un modelo de calidad que permite caracterizar y evaluar la calidad de productos software y sistemas [7]. Aunque el estándar no sustituye a ISO/IEC/IEEE 29148 como marco específico de ingeniería de requisitos, sus características de calidad pueden utilizarse para orientar la definición de requisitos no funcionales y atributos de calidad dentro de una Especificación de Requisitos de Software [3].

Esto significa que las características de calidad relevantes para el producto deben transformarse en requisitos suficientemente claros y evaluables. De esta forma, el ERS actúa como vínculo entre las necesidades de calidad identificadas y las posteriores actividades de diseño, implementación y pruebas.

La importancia de definir correctamente estos atributos está respaldada por la investigación reciente sobre calidad de requisitos, que encuentra una diversidad considerable de atributos y problemas utilizados para estudiar empíricamente la calidad de los requisitos, lo que evidencia la necesidad de utilizar marcos estructurados para definir y evaluar sus propiedades [1].

2.1.5. SWEBOK v4.0 y Pohl (2025) Parte V

El *Guide to the Software Engineering Body of Knowledge (SWEBOK)* v4.0, publicado por IEEE Computer Society en 2024, consolida el conocimiento generalmente aceptado dentro de la ingeniería de software. Su área dedicada a requisitos contempla fundamentos, elicitación, análisis y otras actividades necesarias para transformar necesidades de los interesados en requisitos que puedan ser utilizados durante el desarrollo del software [8, 9]. La versión 4.0 actualiza además el cuerpo de conocimiento para incorporar prácticas contemporáneas de ingeniería de software.

Complementariamente, la segunda edición de *Requirements Engineering: Fundamentals, Principles, and Techniques*, publicada por Springer en 2025, presenta un tratamiento sistemático de la ingeniería de requisitos. Particularmente, la Parte V, denominada *Cross-Sectional Activities*, desarrolla cuatro actividades de validación y tres actividades relacionadas con la gestión de requisitos. Este planteamiento permite comprender ambas actividades como elementos transversales que acompañan la evolución de los requisitos y no únicamente como tareas realizadas al finalizar su especificación [10].

La combinación de SWEBOK v4.0 y Pohl proporciona, por tanto, una perspectiva complementaria al marco normativo: mientras SWEBOK sintetiza el cuerpo de conocimiento aceptado de la disciplina, Pohl profundiza en principios, procedimientos y técnicas específicas para validar y gestionar requisitos.

2.1.6. El CCB según PMBOK 7.ª edición

El control formal de cambios requiere establecer responsabilidades para evaluar las modificaciones propuestas antes de incorporarlas a una línea base. En este contexto se utiliza el Change Control Board (CCB) o Comité de Control de Cambios, entendido como un grupo formalmente establecido al que se asigna autoridad para evaluar determinadas solicitudes de cambio conforme al procedimiento definido para el proyecto [11].

Aplicado a la ingeniería de requisitos, una solicitud de modificación debe registrarse y someterse a análisis antes de modificar directamente el ERS o su línea base. El análisis de impacto permite determinar qué requisitos, artefactos, componentes, costos, plazos o pruebas podrían verse afectados. Con esa información puede adoptarse una decisión sobre la aprobación, rechazo o aplazamiento del cambio y, cuando corresponda, actualizar los elementos afectados manteniendo su trazabilidad [11].

2.2. Validación de requisitos

La validación de requisitos constituye una actividad esencial de la ingeniería de requisitos orientada a determinar si la especificación representa adecuadamente las necesidades, expectativas y objetivos de los interesados. Su importancia radica en detectar problemas antes de que estos se propaguen hacia etapas posteriores del desarrollo, donde su corrección puede implicar modificaciones en el diseño, implementación y pruebas del sistema.

La literatura reciente continúa considerando la validación como una de las actividades fundamentales de la ingeniería de requisitos, señalando que busca garantizar que la información recopilada se encuentre correctamente organizada para satisfacer los objetivos del sistema, evaluando aspectos como exactitud, completitud y corrección de los documentos o modelos de requisitos [2].

Esta actividad se relaciona directamente con la calidad del ERS. Entre los atributos estudiados con mayor frecuencia en la calidad de requisitos se identifican la ambigüedad, completitud, consistencia y corrección, características cuya evaluación permite descubrir deficiencias antes de utilizar la especificación como base para las siguientes etapas del desarrollo [1].

2.2.1. Verificación vs. validación: diferencias conceptuales

Aunque los términos verificación y validación suelen aparecer conjuntamente, representan objetivos diferentes dentro del aseguramiento de la calidad. La verificación se concentra principalmente en comprobar si los requisitos han sido especificados de manera adecuada, es decir, si presentan las propiedades necesarias para poder ser comprendidos, analizados y posteriormente comprobados. En este proceso pueden examinarse características como consistencia, completitud, ausencia de ambigüedad y verificabilidad.

La validación, en cambio, se orienta hacia la correspondencia entre los requisitos documentados y las necesidades reales de los stakeholders. Por tanto, un requisito puede estar correctamente redactado desde el punto de vista formal y aun así no representar lo que necesita el usuario. La validación requiere entonces involucrar a los interesados para determinar si lo especificado describe el sistema que realmente se pretende desarrollar.

La diferencia puede sintetizarse en dos preguntas: la verificación analiza si los requisitos están correctamente especificados, mientras que la validación determina si se han especificado los requisitos adecuados. Sin embargo, ambas actividades son complementarias y contribuyen conjuntamente al aseguramiento de la calidad del ERS.

La importancia de la verificabilidad se evidencia en enfoques destinados específicamente a verificar requisitos de rendimiento y generar entornos de prueba a partir de ellos, lo cual muestra la relación existente entre requisitos correctamente formulados y su posterior comprobación [12].

2.2.2. Principios de validación de requisitos

La validación debe realizarse de manera sistemática y considerar tanto las características individuales de cada requisito como la consistencia del conjunto completo. Un requisito aislado puede parecer adecuado, pero generar contradicciones cuando se relaciona con otros elementos del ERS. Por ello, la revisión debe considerar la especificación desde una perspectiva integral.

Entre los aspectos fundamentales se encuentran la corrección, para determinar si el requisito representa una necesidad real; la completitud, para comprobar que la información necesaria haya sido especificada; la consistencia, para identificar contradicciones entre requisitos; y la ausencia de ambigüedad, de modo que un requisito no admita interpretaciones incompatibles.

Estos aspectos están respaldados por el estudio sistemático de Montgomery et al., que identifica doce atributos de calidad estudiados empíricamente y encuentra que ambigüedad, completitud, consistencia y corrección se encuentran entre los más prominentes en la investigación sobre calidad de requisitos [1].

La validación tampoco debe concebirse exclusivamente como una actividad final. Los requisitos evolucionan y pueden ser modificados como resultado de nuevas necesidades o de la detección de inconsistencias. Por ello, resulta conveniente realizar actividades de validación de manera iterativa durante su elaboración y después de cambios relevantes.

2.2.3. Técnicas de validación

La validación de requisitos puede realizarse mediante diferentes técnicas cuya selección depende de las características del proyecto, los artefactos disponibles y la participación de los stakeholders. Entre las alternativas se encuentran las revisiones e inspecciones, el uso de prototipos, la construcción y evaluación de escenarios, la utilización de listas de comprobación y la derivación temprana de casos o criterios de prueba.

Las revisiones permiten que diferentes participantes examinen los requisitos para localizar omisiones, contradicciones, ambigüedades o información incorrecta. Los prototipos permiten representar anticipadamente determinadas características o interacciones del sistema, facilitando que los usuarios evalúen si el comportamiento esperado corresponde con sus necesidades. Los escenarios, por su parte, describen situaciones concretas de utilización y permiten analizar cómo los requisitos se manifiestan durante una interacción determinada.

La investigación reciente demuestra que las prácticas de validación continúan evolucionando según el contexto tecnológico. Un estudio de mapeo sobre ingeniería de requisitos para sistemas basados en inteligencia artificial identificó 53 estudios relacionados con validación de requisitos, encontrando prácticas como entrevistas, cuestionarios y escenarios, además de nuevas propuestas adaptadas a estos sistemas [13]. La utilización conjunta de diferentes técnicas puede resultar especialmente útil, debido a que una sola técnica difícilmente permite detectar todos los tipos de defectos presentes en una especificación.

2.2.4. Inspección Fagan: origen, fases y roles

La Inspección Fagan constituye un método formal y estructurado de revisión introducido originalmente por Michael Fagan para la detección sistemática de defectos en artefactos de software. Aunque su origen es anterior al intervalo bibliográfico establecido para los artículos de este trabajo, continúa siendo utilizado como referente conceptual en investigaciones contemporáneas relacionadas con inspecciones [14].

Aplicada a un ERS, la inspección permite examinar sistemáticamente los requisitos para identificar defectos antes de avanzar hacia etapas posteriores. El proceso puede organizarse en actividades de planificación, preparación individual, reunión de inspección, corrección o retrabajo y seguimiento. Durante estas actividades se distribuyen responsabilidades entre participantes como el autor del artefacto, el moderador o responsable de conducir la inspección, los inspectores o revisores y, cuando se establece como función independiente, el encargado de registrar los defectos encontrados.

La finalidad principal de la inspección no consiste en rediseñar inmediatamente el requisito durante la reunión, sino en identificar y registrar sus defectos para que posteriormente puedan ser corregidos y verificados.

La vigencia de la inspección de requisitos puede observarse en investigaciones recientes que estudian los fallos dependientes producidos durante la inspección de requisitos, y destacan que la especificación suele ser examinada por un equipo de inspectores para detectar defectos, debido a la influencia que su calidad ejerce sobre las fases posteriores y sobre el producto software [14].

2.2.5. Métricas de inspección

Las métricas de inspección permiten evaluar cuantitativamente los resultados de una revisión y proporcionan información para valorar la efectividad del proceso. Entre las medidas que pueden registrarse se encuentran el número total de defectos identificados, su clasificación por tipo o severidad, el tiempo empleado en la inspección, la cantidad de requisitos examinados y la distribución de defectos entre las diferentes partes del ERS.

A partir de estas medidas pueden construirse indicadores como la densidad de defectos, relacionando la cantidad de defectos encontrados con el tamaño del artefacto inspeccionado. También puede analizarse la productividad de la revisión considerando los defectos identificados respecto al esfuerzo invertido.

No obstante, una cantidad elevada de defectos detectados no necesariamente significa que el proceso de requisitos sea de mejor calidad; puede indicar que la inspección ha sido efectiva, pero también que el documento inicial contenía numerosos problemas. Por ello, las métricas deben interpretarse conjuntamente y compararse entre revisiones o versiones del ERS.

Es necesario además considerar la composición del equipo de inspección, puesto que la incorporación de varios inspectores no garantiza que sus detecciones sean completamente independientes entre sí [14].

2.2.6. Caso de estudio o ejemplo aplicado

Como ejemplo de aplicación, considérese la validación de un requisito perteneciente al proyecto MundiPets:

RF-01: El sistema permitirá registrar una mascota asociándola con los datos correspondientes a su propietario.

Durante una revisión del requisito, el equipo podría identificar que la expresión “datos correspondientes” no especifica cuáles son obligatorios ni establece qué debe ocurrir si el propietario todavía no se encuentra registrado. Además, tampoco indica restricciones sobre la información de la mascota ni las condiciones que determinan que el registro se haya realizado correctamente.

Como resultado de la validación, el requisito podría descomponerse o complementarse con criterios específicos relacionados con los datos obligatorios, las validaciones de entrada y el comportamiento esperado ante situaciones excepcionales. Posteriormente, los stakeholders deberían confirmar que las modificaciones representan correctamente el proceso que desean implementar.

Este ejemplo evidencia que la validación no se limita a comprobar la gramática del requisito. Su objetivo es descubrir información ausente, ambigua, inconsistente o incorrecta antes de que el requisito sea utilizado como fundamento para el diseño y desarrollo. Además, cualquier modificación resultante deberá incorporarse mediante los mecanismos de gestión de requisitos tratados en la Sección 3.3, manteniendo su versión y trazabilidad.

2.3. Gestión de requisitos

La gestión de requisitos comprende el conjunto de actividades destinadas a mantener los requisitos identificados, organizados, actualizados y trazables durante el ciclo de vida del sistema. Su importancia aumenta conforme evoluciona el proyecto, debido a que los requisitos pueden modificarse como consecuencia de nuevas necesidades de los stakeholders, restricciones técnicas, cambios del entorno o decisiones tomadas durante el desarrollo.

Gestionar los requisitos implica, por tanto, mantener información sobre su estado, origen, prioridad, versión y relaciones con otros elementos del proyecto. La trazabilidad constituye uno de los principales mecanismos para conseguirlo, ya que permite establecer vínculos entre los requisitos y los artefactos que los originan o que posteriormente se producen a partir de ellos.

Mantener los requisitos actualizados y comprender su contexto resulta especialmente importante conforme los proyectos aumentan de tamaño y complejidad. La trazabilidad permite responder preguntas relacionadas con el origen de un requisito, los motivos de sus modificaciones y las personas involucradas en su definición [6].

2.3.1. Gestión de la configuración del ERS

La Especificación de Requisitos de Software (ERS) constituye uno de los principales artefactos documentales de la ingeniería de requisitos. Debido a que puede evolucionar durante el proyecto, resulta necesario gestionar su configuración para identificar las versiones existentes y mantener control sobre las modificaciones realizadas.

La gestión de configuración del ERS permite determinar qué versión se encuentra vigente, qué requisitos fueron modificados, cuándo se produjeron los cambios y cuáles son los elementos afectados. De esta manera, se evita que diferentes miembros del equipo trabajen con versiones incompatibles de la especificación y se conserva un historial de su evolución.

Este control adquiere especial importancia cuando los requisitos mantienen relaciones con otros artefactos. Un cambio realizado en el ERS puede repercutir posteriormente en elementos de diseño, código o pruebas, por lo que la configuración y la trazabilidad deben mantenerse coordinadas.

2.3.2. Línea base (baseline) y control de versiones

Una línea base o *baseline* representa una versión formalmente establecida de un conjunto de requisitos que sirve como referencia para continuar el desarrollo. Una vez establecida, sus elementos no deberían modificarse de manera informal; los cambios posteriores deben someterse al procedimiento de control definido por el proyecto.

La línea base no significa que los requisitos sean inmutables. Su propósito consiste en proporcionar un punto de referencia conocido frente al cual puedan identificarse y evaluar las modificaciones posteriores. Así, cuando se aprueba un cambio, se registra la modificación y se genera una nueva versión manteniendo el historial correspondiente.

El control de versiones complementa este mecanismo al permitir identificar la evolución del ERS y de sus requisitos. Como mínimo, debe ser posible conocer qué elemento cambió, cuál era su contenido anterior, cuál es su estado actual y qué modificación dio lugar a la nueva versión.

2.3.3. Atributos de los requisitos

Además de su descripción textual, los requisitos pueden incorporar atributos que proporcionan información adicional para facilitar su gestión. Estos atributos permiten identificar, clasificar, priorizar y controlar los requisitos durante su evolución.

Entre los atributos que pueden asociarse a un requisito se encuentran un identificador único, estado, prioridad, responsable, fuente u origen, versión, fecha de creación o modificación y criterios asociados a su verificación. La selección concreta de atributos dependerá de las necesidades del proyecto y de su proceso de gestión.

Por ejemplo, un requisito funcional podría registrarse de la siguiente manera:

Identificador: RF-01
Descripción: Registrar mascota.
Prioridad: Alta.
Fuente: Stakeholder.
Estado: Aprobado.
Versión: 1.1.

Esta información facilita posteriormente operaciones como localizar requisitos, identificar modificaciones, determinar responsabilidades y establecer relaciones de trazabilidad. Por ello, los atributos no sustituyen al contenido del requisito, sino que proporcionan los metadatos necesarios para administrarlo durante su ciclo de vida.

2.3.4. Trazabilidad: pre-traceability y post-traceability

La trazabilidad de requisitos es la capacidad de establecer y mantener relaciones entre un requisito y la información vinculada con su origen, evolución e implementación. Estas relaciones permiten comprender por qué existe determinado requisito y qué elementos podrían verse afectados si se modifica.

Se distingue entre *pre-requirements specification traceability* (pre-RS) y *post-requirements specification traceability* (post-RS). La primera establece relaciones entre los requisitos y sus fuentes anteriores a su incorporación en la especificación, como entrevistas con stakeholders, reuniones, documentos organizacionales o sistemas existentes [6]. Por ejemplo:

Entrevista con propietario → necesidad identificada → RF-01 Registrar mascota.

La post-traceability, por otra parte, establece relaciones entre los requisitos especificados y los artefactos

generados posteriormente durante el desarrollo, como elementos de diseño, código fuente y casos de prueba. Por ejemplo:

RF-01 Registrar mascota → módulo de registro → código correspondiente → CP-01 Prueba de registro.

Por tanto, ambas perspectivas permiten reconstruir el ciclo del requisito: la pre-traceability ayuda a comprender de dónde proviene, mientras que la post-traceability permite conocer cómo fue desarrollado y comprobado.

La investigación reciente continúa demostrando la importancia y, al mismo tiempo, las dificultades prácticas de mantener estos vínculos, señalando las barreras que dificultan la adopción de trazabilidad en proyectos reales y una menor atención histórica hacia la pre-trazabilidad en comparación con la trazabilidad posterior a la especificación [6, 15].

2.3.5. Matriz de trazabilidad

Una matriz de trazabilidad de requisitos constituye una representación estructurada de las relaciones existentes entre requisitos y otros elementos del proyecto. Su objetivo es facilitar la comprobación de cobertura y permitir identificar rápidamente qué artefactos están relacionados con determinado requisito.

Una representación simplificada para MundiPets podría ser la que se muestra en la Tabla 1.

Tabla 1: Ejemplo simplificado de matriz de trazabilidad para MundiPets.

ID	Requisito	Fuente	Diseño/Módulo	Caso de prueba
RF-01	Registrar mascota	Entrevista E01	Gestión de mascotas	CP-01
RF-02	Consultar mascota	Entrevista E02	Gestión de mascotas	CP-02
RF-03	Actualizar datos	Entrevista E02	Gestión de mascotas	CP-03

La matriz permite realizar tanto seguimiento hacia adelante como hacia atrás. Por ejemplo, partiendo de RF-01 puede localizarse el módulo y caso de prueba relacionados; en sentido contrario, partiendo de CP-01 puede identificarse qué requisito justifica la existencia de dicha prueba.

Esta información también resulta útil durante el análisis de cambios. Si RF-01 se modifica, los enlaces permiten identificar los artefactos potencialmente afectados y determinar cuáles deben revisarse.

No obstante, mantener trazabilidad tiene un costo organizacional. Un estudio empírico encontró que, aunque existen numerosas técnicas y herramientas de trazabilidad, su utilización práctica continúa enfrentándose a distintas barreras [15].

2.3.6. Proceso de cambio: RFC, análisis de impacto, CCB, implementación y cierre

Los requisitos pueden cambiar durante el desarrollo, pero sus modificaciones deben gestionarse de manera controlada cuando forman parte de una línea base. Un procedimiento estructurado evita que una modificación sea incorporada sin conocer previamente sus consecuencias [6].

El proceso puede comenzar mediante una *Request for Change* (RFC) o solicitud de cambio, en la cual se registra la modificación propuesta y su justificación. Posteriormente se realiza un análisis de impacto, considerando los requisitos relacionados y los artefactos que podrían verse afectados, además de factores como esfuerzo, costo, riesgos y planificación [5].

Con esta información, el Change Control Board (CCB) o la autoridad establecida por el proyecto puede evaluar la solicitud y decidir su aprobación, rechazo o aplazamiento. Si el cambio es aprobado, se procede a su implementación, actualizando el requisito y los elementos relacionados. Finalmente se realiza el cierre, verificando que las modificaciones hayan sido efectuadas correctamente, actualizando las versiones y conservando la trazabilidad del cambio.

De manera resumida, el flujo puede representarse como:

RFC → registro → análisis de impacto → evaluación del CCB → aprobación/rechazo → implementación → verificación → actualización de línea base → cierre.

La trazabilidad desempeña un papel fundamental durante este procedimiento porque permite determinar las posibles consecuencias de modificar un requisito. La literatura reciente señala precisamente que los enlaces de trazabilidad ayudan a comprender el contexto y evolución de los requisitos, incluyendo las razones que motivaron determinadas modificaciones [6].

2.3.7. Métricas de volatilidad de requisitos

La volatilidad de requisitos representa el grado en que los requisitos cambian durante un período determinado del proyecto. Su seguimiento permite identificar situaciones en las que se están produciendo modificaciones

frecuentes y analizar si estas pueden afectar la estabilidad del desarrollo.

Entre los elementos que pueden contabilizarse se encuentran los requisitos añadidos, modificados y eliminados durante un período. A partir de estos datos pueden establecerse indicadores relativos respecto al número total de requisitos existentes en la línea base.

Por ejemplo, para fines de seguimiento interno podría utilizarse una razón como:

$$V = \frac{R_a + R_m + R_e}{R_t} \times 100 \quad (1)$$

donde R_a representa los requisitos añadidos, R_m los modificados, R_e los eliminados y R_t el número de requisitos tomado como referencia para el período. Esta expresión debe entenderse como un indicador operativo de seguimiento y no como una fórmula universal establecida por ISO/IEC/IEEE 29148.

Si una línea base contiene 50 requisitos y durante un período se añaden 2, se modifican 3 y se elimina 1, el indicador sería:

$$V = \frac{2 + 3 + 1}{50} \times 100 = 12 \quad (2)$$

El resultado indica que durante ese período se produjo actividad de cambio equivalente al 12 % del conjunto tomado como referencia. Sin embargo, el porcentaje no debe interpretarse aisladamente como bueno o malo; debe analizarse junto con la etapa del proyecto, las causas de los cambios y sus impactos.

Un estudio que aborda específicamente la *Requirement Change Volatility* (RCV) señala que una planificación insuficiente frente a cambios destinados a responder a las expectativas de los stakeholders puede producir propagación de modificaciones y contribuir a problemas en el proyecto. Su investigación utiliza redes de requisitos y técnicas de aprendizaje automático para evaluar y predecir esta volatilidad [16].

De esta manera, las métricas de volatilidad complementan la gestión de cambios: mientras la RFC y el análisis de impacto permiten administrar una modificación concreta, las métricas permiten observar el comportamiento agregado de los cambios a lo largo del proyecto.

2.4. Herramientas CASE para IR

Las herramientas CASE (*Computer-Aided Software Engineering*) constituyen el soporte tecnológico que automatiza, total o parcialmente, las actividades de elicitación, especificación, verificación, validación y gestión de los requisitos a lo largo del ciclo de vida del software. Su adopción responde a la necesidad de reducir el esfuerzo manual asociado a la trazabilidad, el control de versiones y la comunicación entre los interesados, actividades que, gestionadas sin apoyo automatizado, incrementan la probabilidad de inconsistencias en el ERS [3].

2.4.1. Clasificación de herramientas CASE

Las herramientas CASE aplicadas a la ingeniería de requisitos (IR) se clasifican habitualmente según la etapa del proceso que automatizan y el alcance funcional que ofrecen. Un análisis reciente de 56 herramientas de IR agrupó sus características en siete perspectivas complementarias: gestión de proyectos, especificación, colaboración, personalización, interoperabilidad, metodología y soporte al usuario, evidenciando que ninguna herramienta cubre de manera uniforme la totalidad de estas dimensiones [17]. Desde una perspectiva funcional más orientada al mercado, otros estudios distinguen entre herramientas de *requirements gathering* (apoyadas en técnicas visuales, diagramas de contexto, descomposición funcional e historias de usuario) y herramientas de *requirements management* propiamente dichas, orientadas al control de cambios, la trazabilidad y la generación de reportes [18]. Esta doble clasificación por etapa del proceso y por alcance funcional resulta coherente con el proceso de validación y gestión descrito en el estándar ISO/IEC/IEEE 29148:2018 y con los procesos de control de cambios de la ISO/IEC/IEEE 12207:2017 [3, 5].

2.4.2. Funcionalidades esperables en gestión de requisitos

Con independencia de la clasificación adoptada, la literatura converge en un conjunto de funcionalidades mínimas esperables en una herramienta CASE orientada a la gestión de requisitos. Ozkaya et al. [17] identifican, entre las más recurrentes, la especificación estructurada de requisitos, el versionado, la trazabilidad bidireccional, el soporte a la colaboración distribuida entre analistas y stakeholders, y la interoperabilidad con otras herramientas del ciclo de vida (gestión de pruebas, control de código y gestión de proyectos). En la misma línea, la comparación cualitativa de diez herramientas comerciales de gestión de requisitos realizada por Nadeem et al. [18] evalúa explícitamente diez parámetros –seguimiento de errores, gestión jerárquica de requisitos, diseño

visual, gestión de riesgos, gestión de pruebas, verificación de requisitos, análisis de calidad, versionado, gestión de *releases* y generación de reportes–, concluyendo que ninguna herramienta analizada obtiene un desempeño uniformemente alto en todos ellos, lo que refuerza la necesidad de un criterio de selección alineado con las prioridades específicas del proyecto.

2.4.3. Criterios de selección de una herramienta CASE

La selección de una herramienta CASE no es un problema trivial: involucra criterios técnicos, organizacionales y económicos que deben ponderarse según el contexto del proyecto. Bjarnason et al. [19] proponen un modelo de selección de software a gran escala construido a partir de revisiones rápidas e iterativas, que combina una lista de criterios de evaluación (funcionales, de usabilidad, de soporte del proveedor y de costo total de propiedad) con un proceso estructurado para identificar, valorar y filtrar alternativas antes de la adopción definitiva. Este enfoque es aplicable directamente a la selección de herramientas de gestión de requisitos, dado que ambos dominios comparten la dificultad de comparar productos con conjuntos de características parcialmente solapados y una oferta comercial heterogénea. Entre los criterios más citados en la literatura destacan la facilidad de integración con el resto del ecosistema de herramientas del proyecto, la curva de aprendizaje, el soporte a estándares de la industria y la escalabilidad para equipos distribuidos [17, 19].

2.4.4. Limitaciones y costos

La adopción de herramientas CASE no está exenta de limitaciones. En primer lugar, el costo total de propiedad –licenciamiento, capacitación del equipo y mantenimiento– constituye una barrera relevante, particularmente para organizaciones pequeñas o proyectos con presupuestos ajustados [19]. En segundo lugar, el análisis comparativo de Nadeem et al. [18] evidencia que las herramientas orientadas a funcionalidades avanzadas (gestión de riesgos, análisis de calidad de requisitos) suelen sacrificar simplicidad de uso, lo que puede traducirse en una curva de adopción prolongada. Finalmente, Ozkaya et al. [17] señalan que la interoperabilidad entre herramientas de distintos proveedores continúa siendo una limitación estructural del mercado, lo que obliga frecuentemente a las organizaciones a asumir integraciones a medida o a tolerar silos de información entre la especificación de requisitos y otras disciplinas del ciclo de vida (pruebas, arquitectura, gestión de proyectos).

2.5. IR en procesos ágiles

La incorporación de prácticas ágiles ha transformado sustancialmente la forma en que se elicitán, documentan y gestionan los requisitos, desplazando el énfasis desde la especificación exhaustiva y previa hacia una construcción incremental y colaborativa del entendimiento del producto. Esta sección revisa los artefactos, prácticas y mecanismos de trazabilidad propios de la IR ágil, y los contrasta con el enfoque documental descrito en 3.1–3.3.

2.5.1. Product backlog, historias de usuario y criterios de aceptación

El *product backlog* constituye el artefacto central de la gestión ágil de requisitos: una lista ordenada y viva de necesidades del producto que se refina de manera continua. Su unidad de trabajo predominante es la historia de usuario, cuyo origen se remonta a la programación extrema y que se popularizó como mecanismo para capturar requisitos desde la perspectiva del usuario final, priorizando la conversación sobre la documentación exhaustiva [20]. No obstante, la sola redacción de una historia de usuario no garantiza su calidad: el estudio de caso de Kuhail y Lauesen [21], que evaluó las respuestas de ocho profesionales de TI frente a un proyecto real utilizando los criterios de calidad de la IEEE 830 (completitud, corrección, verificabilidad y trazabilidad), encontró que las historias de usuario producidas cubrieron apenas el 33 % de las necesidades identificadas en el informe de análisis original, lo que evidencia el riesgo de pérdida de información al migrar de una especificación documental a un formato ágil sin mecanismos de control adicionales. Por ello, Ferreira et al. [20] proponen enriquecer las historias de usuario con criterios de aceptación explícitos, épicas y temas que permitan preservar la trazabilidad conceptual entre el requisito de alto nivel y su implementación incremental, en línea con los atributos de calidad definidos en la ISO/IEC/IEEE 29148:2018 [3].

2.5.2. Grooming y Definition of Ready / Definition of Done

El refinamiento del backlog (*backlog grooming* o *refinement*) es la actividad continua mediante la cual el equipo desglosa, estima y clarifica los elementos del backlog hasta que cumplen un nivel de detalle suficiente para ser abordados en un sprint. La Guía de Scrum no define formalmente una *Definition of Ready* (DoR), pero establece que los elementos del backlog deben alcanzar un grado de transparencia y comprensión que permita completarlos dentro de un sprint [22]. La *Definition of Done* (DoD), en cambio, sí forma parte del marco oficial de Scrum y constituye un compromiso formal y compartido por el equipo sobre lo que significa que un incremento esté terminado, siendo responsabilidad primaria del equipo de desarrollo mantener su cumplimiento [22]. Tomaselli et al. [23], en su revisión sistemática sobre métodos de gestión de requisitos en desarrollo ágil, destacan que

la ausencia de criterios de entrada y salida explícitos –es decir, de un DoR y un DoD bien definidos– es una de las causas recurrentes de retrabajo y de defectos de calidad en los incrementos entregados, y recomiendan su integración temprana como práctica de gestión de requisitos dentro del marco de calidad ágil (*Agile Quality Framework*).

2.5.3. BDD y formato Gherkin

El desarrollo guiado por comportamiento (*Behavior-Driven Development*, BDD) extiende la lógica de las historias de usuario hacia especificaciones ejecutables, escritas en lenguaje natural estructurado mediante el formato Gherkin (*Given-When-Then*), que permite a analistas, desarrolladores y evaluadores compartir un mismo artefacto como documentación, criterio de aceptación y base para pruebas automatizadas. La revisión sistemática de Abushama et al. [24] sobre el efecto del desarrollo guiado por pruebas (TDD) y del BDD en los factores de éxito de un proyecto concluye que ambas prácticas inciden positivamente en la satisfacción del cliente y en la reducción de defectos, atribuyendo al BDD un valor adicional en la mejora de la comunicación entre perfiles técnicos y no técnicos gracias a la legibilidad de sus escenarios. Esta característica convierte al formato Gherkin en un mecanismo natural de validación de requisitos en entornos ágiles, ya que cada escenario actúa simultáneamente como criterio de aceptación verificable y como evidencia de trazabilidad entre la necesidad del negocio y el comportamiento implementado del sistema.

2.5.4. Trazabilidad en entornos ágiles

La trazabilidad de requisitos –la capacidad de seguir un requisito desde su origen hasta su implementación y verificación– presenta desafíos particulares en contextos ágiles, donde la reducción deliberada de la documentación formal puede debilitar los enlaces explícitos entre artefactos. El estudio de mapeo de Charalampidou et al. [25] sobre trazabilidad de software identifica que, si bien la mayoría de los métodos de trazabilidad fueron concebidos originalmente para procesos secuenciales y documentales, existe una tendencia creciente hacia enfoques ligeros y semiautomatizados basados en recuperación de información y en la vinculación de commits, historias de usuario y casos de prueba que se adaptan mejor a la cadencia iterativa del desarrollo ágil. En la misma dirección, Loh Mun Keong y Che Embi [26] advierten que la documentación de requisitos no funcionales es una de las prácticas más frecuentemente descuidadas en equipos ágiles, lo que compromete la trazabilidad de atributos de calidad como los definidos en la ISO/IEC 25010:2023 [7] y exige mecanismos complementarios (etiquetado de historias, matrices ligeras o herramientas CASE integradas) para no perder dicha trazabilidad frente a la mayor velocidad de entrega.

2.5.5. IR documental frente a IR ágil: ¿cuándo conviene cada enfoque?

La disyuntiva entre un enfoque documental –propio de los estándares revisados en 3.1 a 3.3, con énfasis en el ERS, la línea base y la matriz de trazabilidad formal– y un enfoque ágil –centrado en el backlog vivo, las historias de usuario y la trazabilidad ligera– no debe entenderse como una elección excluyente, sino como una decisión de diseño de proceso condicionada por el contexto del proyecto. La revisión de Tomaselli et al. [23] concluye que los proyectos con alta variabilidad de requisitos, plazos cortos y participación constante del cliente se benefician de prácticas ágiles de gestión de requisitos, mientras que proyectos regulados, de misión crítica o con requisitos contractuales estrictos –por ejemplo, aquellos sujetos a normas como la ISO/IEC/IEEE 29148:2018 o a certificaciones sectoriales– requieren preservar un nivel de documentación formal y trazabilidad verificable que los formatos puramente ágiles no garantizan por defecto [3]. En la práctica, numerosas organizaciones adoptan esquemas híbridos: mantienen una especificación de alto nivel documentada y una línea base contractual, mientras gestionan el detalle operativo mediante backlog, historias de usuario y escenarios Gherkin, combinando así la trazabilidad formal exigida por el marco normativo con la agilidad necesaria para responder a los cambios propios del desarrollo iterativo [23, 26].

3. Inspección formal del ERS (método Fagan)

3.1. Planificación y roles

Debido a que el equipo está conformado por tres integrantes, se asignaron los cuatro roles exigidos por el método Fagan de la siguiente manera: Andy Johel Mendoza Párraga asumió simultáneamente los roles de Moderador y Lector, mientras que Winston Damian Cedeño Ávila y Mayra Lucila Córdova Carreño actuaron como Inspector 1 e Inspector 2, respectivamente. Esta asignación quedó formalizada en el documento *Planificación de la Inspección Formal Fagan*, firmado por los tres integrantes el 7 de agosto de 2026.

El alcance de la inspección se delimitó a las secciones 2 (Descripción general), 3 (Requisitos específicos completos) y 5 (Priorización y trazabilidad extendida) del ERS_SRS_2A_v1.0, por concentrar la mayor densidad de requisitos verificables y de relaciones de trazabilidad del documento. Dicho alcance corresponde a un total de **50 páginas efectivas** (7 páginas de la Sección 2, 34 de la Sección 3 y 9 de la Sección 5), según el conteo realizado sobre el PDF compilado del ERS.

La reunión de inspección se planificó para el 8 de agosto de 2026, a las 16:00, en modalidad virtual (Google Meet), con una duración máxima acordada de 90 minutos.

3.2. Preparación individual

Previo a la reunión, cada integrante realizó de forma independiente una revisión del ERS dentro del alcance definido, registrando los defectos encontrados en su propia copia del Anexo A, con marca temporal de inicio y fin de la preparación. La Tabla 2 resume la evidencia de esta etapa.

Tabla 2: Evidencia de preparación individual por integrante.

Integrante	Rol	Horario de preparación	Defectos detectados
Andy Mendoza	Moderador / Lector	08/08/2026, 09:00–11:15	6
Winston Cedeño	Inspector 1	08/08/2026, 10:00–11:30	6
Mayra Córdova	Inspector 2	08/08/2026, 10:15–11:30	6

Cada inspector superó el mínimo de 6 defectos exigido, evaluando las seis propiedades de calidad establecidas: ambigüedad, completitud, consistencia, verificabilidad, trazabilidad y factibilidad. Los defectos identificados abarcaron desde imprecisiones de redacción (por ejemplo, el uso de “radio ≥ 1 km” en RNF-16, que permite valores arbitrariamente grandes) hasta brechas estructurales más profundas, como la ausencia de un Requisito Funcional que implemente el derecho de supresión de cuenta exigido por RL-04.

3.3. Acta de la reunión de inspección

La reunión se realizó el 8 de agosto de 2026, a partir de las 16:00, con una duración aproximada de 60 minutos, dentro del límite máximo de 90 minutos establecido. Participaron los tres integrantes del equipo, cada uno en su rol asignado. Conforme a la regla acordada en la planificación, la reunión se limitó estrictamente a la detección y el registro de defectos, sin discutir ni proponer soluciones.

Durante la reunión, el Lector guio la revisión de las secciones 2, 3 y 5 del ERS, y cada Inspector reportó los defectos identificados en su preparación individual. El Moderador consolidó la totalidad de los defectos reportados en el Anexo B en tiempo real. Como resultado, se registraron **18 defectos únicos**, distribuidos equitativamente entre los tres integrantes (6 defectos detectados por cada uno), superando ampliamente el mínimo de 15 defectos exigido, con 2 defectos críticos y 7 mayores (9 en total), por encima del mínimo de 3 críticos o mayores establecido por la guía.

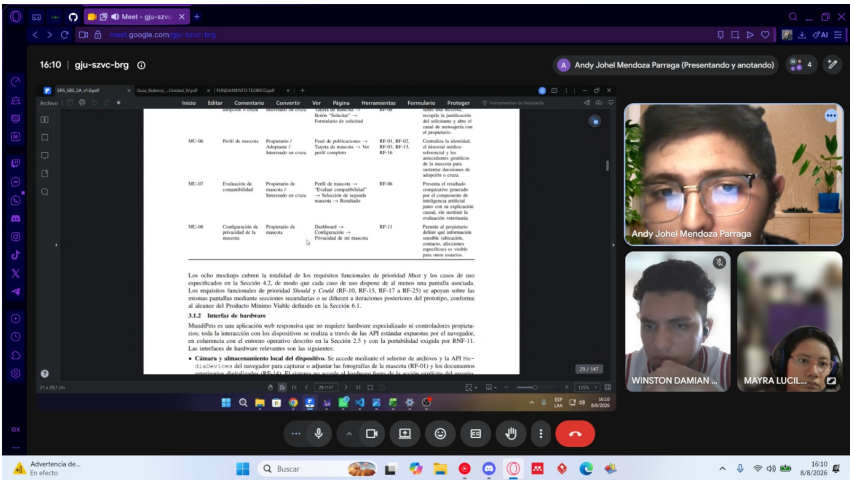


Figura 1: Evidencia fotográfica de la reunión de inspección Fagan (8 de agosto de 2026).

3.4. Métricas de inspección

A partir de la información registrada en el Anexo B y en los Anexos A individuales, se calcularon las métricas de la Tabla 3.

Tabla 3: Métricas de la inspección formal Fagan.

Métrica	Valor
Defectos totales detectados	18
Páginas efectivas del alcance	50
Densidad de defectos	0.36 defectos/página
Esfuerzo de preparación individual	5.00 horas-persona
Esfuerzo de la reunión (3 personas × 60 min)	3.00 horas-persona
Esfuerzo total de la inspección	8.00 horas-persona
Productividad de la inspección	2.25 defectos/hora-persona
Tasa de detección por inspector	33.3 % cada uno (6/18)

Tabla 4: Distribución de defectos por tipo (propiedad de calidad).

Tipo	Cantidad
Compleitud	6
Consistencia	5
Ambigüedad	3
Verificabilidad	2
Trazabilidad	1
Factibilidad	1

Tabla 5: Distribución de defectos por severidad.

Severidad	Cantidad
Crítico	2
Mayor	7
Menor	9

La densidad de defectos (0.36 por página) indica que, en promedio, se detectó poco más de un defecto cada tres páginas del alcance inspeccionado, lo cual es consistente con un ERS que ya había pasado por un ciclo previo de especificación (Entrega 2A) pero que aún conserva inconsistencias entre secciones. La distribución por tipo muestra que **completitud** (6) y **consistencia** (5) concentran más del 60 % de los defectos, lo que sugiere que el principal riesgo del documento no es la redacción individual de cada requisito, sino la coherencia entre secciones y artefactos relacionados (por ejemplo, discrepancias entre la matriz de trazabilidad y la clasificación de requisitos, o entre un RF y su historia de usuario asociada). La distribución por severidad, con 9 defectos críticos o mayores sobre 18 totales (50 %), confirma que la mitad de los hallazgos requiere corrección obligatoria antes

de establecer la nueva línea base del ERS. Finalmente, la tasa de detección equilibrada entre los tres inspectores (33.3 % cada uno) indica que ningún integrante concentró desproporcionadamente la carga de revisión, aunque –tal como advierte la literatura citada en la Sección 2.2.5– esto no garantiza por sí solo independencia total entre las detecciones, ya que varios defectos fueron reportados sobre secciones distintas del ERS por cada inspector.

4. Correcciones aplicadas

Conforme a la meta de calidad establecida para esta práctica, se corrigió el 100 % de los defectos clasificados como críticos o mayores (9 de 18), documentando en cada caso el texto original (*antes*) y la redacción corregida (*después*). Los 9 defectos menores restantes se abordan en la Sección 4.2.

4.1. Corrección de defectos críticos y mayores

Defecto 4 (Crítico) — Inconsistencia numérica en la matriz de trazabilidad.

Antes: “La matriz completa, con sus 40 filas (TR-01 a TR-40), se presenta en el Apéndice D [...]. La matriz cubre los 50 elementos especificados [...] de las 50 filas...”

Después: “La matriz completa, con sus 50 filas (TR-01 a TR-50), se presenta en el Apéndice D, cubriendo la totalidad de los 50 elementos especificados (25 RF, 16 RNF y 9 RD).”

Se unificó el número de filas de la matriz (50) en todas las menciones de la Sección 5.5, eliminando la referencia residual a “40 filas”. *Requisito afectado:* Sección 5.5 (matriz de trazabilidad).

Defecto 5 (Trazabilidad, Crítico) — Ausencia de RF que implemente RL-04/RD-06.

Antes: RD-06 (Mecanismo de eliminación de cuenta y datos personales) reconoce en su propia justificación que “ningún RF de los 25 definidos cubría esta función”.

Después: Se incorpora el nuevo **RF-26 — Eliminar cuenta y datos personales**, que implementa RD-06: permite al propietario solicitar la eliminación de su cuenta, ejecuta la supresión o anonimización de sus datos personales dentro de un plazo definido, y deja registro de auditoría de la solicitud.

Esta corrección se formaliza mediante RFC-01 (Sección 5.1), por tratarse de un cambio de alcance que incorpora un nuevo requisito funcional. *Requisitos afectados:* RL-04, RD-06, RF-26 (nuevo).

Defecto 2 (Mayor) — Falta de dato de verificación profesional en RF-12.

Antes: RF-12 exige “verificar su identidad” mediante “información requerida”, sin especificar qué dato valida la identidad profesional del rol Veterinaria.

Después: Se añade a las Entradas de RF-12 el campo *número de registro profesional ante el ente regulador competente*, exigido únicamente cuando el rol seleccionado sea Veterinaria, y se referencia explícitamente en el flujo de CU-09.

Requisitos afectados: RF-12, CU-09.

Defecto 3 (Mayor) — Dependencia bloqueante entre RF-24 (Should) y CU-02 (Must).

Antes: RF-24 (trazabilidad de cepa, lote y aplicador de cada vacuna) tiene prioridad *Should*, pero el paso 4 de CU-02 (*Must*) exige registrar esos mismos datos como parte obligatoria del flujo.

Después: Se eleva la prioridad de RF-24 a *Must*, alineándola con la dependencia real que impone CU-02, evitando que un caso de uso obligatorio dependa de un requisito de prioridad inferior.

Requisitos afectados: RF-24, CU-02.

Defecto 6 (Mayor) — RNF-02 no medible en el MVP.

Antes: RNF-02 exige disponibilidad “ ≥ 99 % mensual” sin distinguir el entorno de aplicación, mientras que el MVP se declara como aplicación local sin servidor (`localStorage`).

Después: Se añade a RNF-02 la aclaración “Este criterio aplica al entorno de producción con backend desplegado; no es exigible ni medible sobre el MVP local descrito en la Sección 6.”

Requisitos afectados: RNF-02.

Defecto 9 (Mayor) — Actor “Refugio de animales” ausente en RF-05.

Antes: La Tabla 5 (Clases y características de usuarios) asigna al Refugio de animales la función de “revisar solicitudes y comunicarse con adoptantes”, pero RF-05 (Gestionar solicitudes de adopción) solo lista como actores a “Adoptante, Propietario de mascota”.

Después: Se añade “Refugio de animales” a la lista de actores de RF-05, y se incorpora al flujo principal el caso en que la solicitud sea gestionada por un refugio en representación de las mascotas bajo su custodia.

Requisitos afectados: RF-05, Tabla 5.

Defecto 10 (Mayor) — Entrada “fecha de aplicación” faltante en RF-02.

Antes: RF-02 (Gestionar historial médico) no lista “fecha de aplicación” como entrada explícita para el registro de vacunas, aunque HU-02/CA-02 exige ese dato y genera un error si se omite.

Después: Se agrega “fecha de aplicación” a las Entradas de RF-02, como campo obligatorio para todo registro de tipo vacuna, alineando el requisito funcional con su historia de usuario e criterio de aceptación asociados.

Requisitos afectados: RF-02, HU-02, CA-02.

Defecto 15 (Mayor) — Contradicción sobre protección del historial médico (RL-07).

Antes: La definición de “titular” (Sección 2) establece que el historial médico de la mascota no constituye, en sí mismo, un dato personal protegido, mientras que RL-07 afirma que “el historial clínico de la mascota se protege como extensión indirecta de esos datos”.

Después: Se reformula la definición de “titular” en la Sección 2 para reconocer explícitamente que, si bien la mascota no es titular de datos personales, su historial clínico queda protegido de forma indirecta cuando permite identificar datos del propietario, en concordancia con RL-07.

Requisitos afectados: RL-07, RNF-01, RNF-05, definición de “titular” (Sección 2).

Defecto 13 (Mayor) — Matriz de trazabilidad no cubre los requisitos legales.

Antes: La clasificación de requisitos (Sección 5.1) declara 59 requisitos vigentes (25 RF + 16 RNF + 9 RD + 9 RL), pero la matriz de trazabilidad (Sección 5.5) declara cubrir solo “los 50 elementos especificados” (25 RF + 16 RNF + 9 RD), sin incluir los 9 RL.

Después: Se amplía la matriz de trazabilidad para incluir los 9 requisitos legales (RL-01 a RL-09), enlazándolos hacia las restricciones de diseño (RD) que los implementan, alcanzando así los 68 elementos trazados en total (25 RF + 16 RNF + 9 RD + 9 RL + 9 enlaces RL→RD).

Requisitos afectados: Sección 5.1, Sección 5.5, RL-01 a RL-09.

4.2. Defectos menores

Los 9 defectos clasificados como menores también fueron corregidos en su totalidad, dado que su resolución no comprometía el alcance ni el cronograma de esta práctica. El detalle completo de cada corrección (antes/después) se encuentra documentado en el AnexoB_registro_defectos.xlsx, columna *Corrección aplicada*, marcada como “Sí” para los 18 defectos registrados.

5. Gestión del cambio y Change Control Board (CCB)

Como resultado de la inspección formal (Sección 3) se identificaron oportunidades de cambio que exceden la simple corrección de redacción y que requieren ser tramitadas mediante el procedimiento formal de control de cambios descrito en la Sección 2.1.6. A continuación se documentan las tres solicitudes de cambio (RFC) sometidas al Change Control Board (CCB), su análisis de impacto y la decisión adoptada.

5.1. RFC-01 – Cambio de alcance (nuevo requisito)

ID	RFC-01
Fecha	09/08/2026
Solicitante y rol	Winston Cedeño Ávila, Inspector 1
Requisito afectado	RL-04, RD-06
Descripción del cambio	Incorporar un nuevo requisito funcional, RF-26 — Eliminar cuenta y datos personales , que implemente el mecanismo de eliminación exigido por RD-06 y por el derecho de supresión reconocido en RL-04 (Art. 15 LOPDP).
Justificación de negocio	La inspección detectó (Defecto 5, crítico) que RD-06 reconoce en su propia justificación que “ningún RF de los 25 definidos cubría esta función”. Sin este requisito, el sistema incumpliría un derecho legal reconocido explícitamente en el propio ERS, exponiendo al PFC a un riesgo de cumplimiento normativo.
Requisitos impactados	RL-04, RD-06, RNF-01 (protección de datos), RNF-05 (integridad de la información)
Artefactos impactados	Modelo de datos (eliminación en cascada del historial médico y publicaciones), módulo de cuentas, matriz de trazabilidad (Sección 5.5)
Costo estimado	6 horas-persona (diseño del flujo, implementación del RF y actualización de la matriz de trazabilidad)
Riesgo	Medio. La eliminación en cascada del historial médico puede afectar la trazabilidad de datos ya compartidos con veterinarias o refugios; requiere definir una política de retención antes de implementar.
Recomendación técnica	Aprobar. El cambio cierra un vacío ya reconocido por el propio ERS y es indispensable para el cumplimiento del Art. 15 de la LOPDP.

Tabla 6: RFC-01 – Cambio de alcance.

5.2. RFC-02 – Cambio de calidad (modificación de un RNF)

ID	RFC-02
Fecha	09/08/2026
Solicitante y rol	Mendoza Párraga Andy Johel, Moderador
Requisito afectado	RNF-02
Descripción del cambio	Acotar el alcance de RNF-02 (disponibilidad ≥ 99 % mensual) para que aplique exclusivamente al entorno de producción con backend desplegado, aclarando que no es exigible ni medible sobre el MVP local descrito en la Sección 6.
Justificación de negocio	La inspección detectó (Defecto 6, mayor) que el MVP se declara como aplicación local sin servidor (<code>localStorage</code>), lo que vuelve el criterio de disponibilidad mensual no medible en esta iteración. Mantener el requisito sin acotar genera una expectativa de verificación imposible de cumplir en el MVP.
Requisitos impactados	RNF-02, Sección 6 (Producto Mínimo Viable)
Artefactos impactados	Documento ERS (redacción de RNF-02), plan de pruebas de aceptación del MVP
Costo estimado	1 hora-persona (ajuste de redacción y revisión de consistencia con la Sección 6)
Riesgo	Bajo. El cambio no altera funcionalidad, solo delimita el alcance de verificación de un RNF ya existente.
Recomendación técnica	Aprobar. Evita un criterio de verificación no aplicable al MVP y mantiene el requisito vigente para la fase de producción.

Tabla 7: RFC-02 – Cambio de calidad.

5.3. RFC-03 – Cambio de restricción o normativa

ID	RFC-03
Fecha	09/08/2026
Solicitante y rol	Córdova Carreño Mayra Lucila, Inspector 2
Requisito afectado	RD-08, RF-10
Descripción del cambio	Formalizar en RF-10 la exigencia de un canal de respaldo <i>in-app</i> obligatorio ante la indisponibilidad del proveedor externo de mensajería (WhatsApp Business API), actualmente mencionado únicamente en el campo “Impacto” de RD-08 sin quedar reflejado como comportamiento exigible del requisito funcional.
Justificación de negocio	RD-08 reconoce que la disponibilidad del canal de recordatorios “queda condicionada a las políticas y disponibilidad de dicho proveedor” y menciona la necesidad de un canal de respaldo, pero RF-10 no exige explícitamente ese respaldo como parte de su comportamiento. Sin esta restricción formalizada, una caída del proveedor externo dejaría a los propietarios sin recordatorios de controles preventivos, afectando directamente el objetivo sanitario del sistema.
Requisitos impactados	RF-10, RD-08, RNF-09 (compatibilidad)
Artefactos impactados	Descripción y criterio de verificación de RF-10, arquitectura de integración externa, matriz de trazabilidad
Costo estimado	3 horas-persona (actualización de RF-10 y su criterio de verificación, prueba del canal de respaldo)
Riesgo	Bajo. El canal de respaldo (notificación in-app) ya forma parte de la arquitectura declarada en la Sección 2 (entorno operativo); el cambio solo formaliza su exigibilidad en el requisito funcional.
Recomendación técnica	Aprobar. Cierra una brecha entre lo declarado como mitigación de riesgo (RD-08) y lo exigido funcionalmente (RF-10).

Tabla 8: RFC-03 – Cambio de restricción.

5.4. Acta del CCB

Fecha: 9 de agosto de 2026. **Modalidad:** Virtual (Google Meet). **Duración:** 45 minutos.

Integrante	Rol en el CCB
Mendoza Párraga Andy Johel	Presidente del CCB
Cedeño Ávila Winston Damian	Analista de requisitos
Córdova Carreño Mayra Lucila	Representante del cliente / Desarrolladora

Tabla 9: Asistentes al CCB y roles asignados.

Agenda: revisión y decisión sobre RFC-01, RFC-02 y RFC-03, en ese orden.

Deliberación. Para RFC-01, el CCB coincidió en que la ausencia de un RF que implemente RD-06 constituye un vacío crítico ya autodocumentado por el propio ERS, y que su riesgo medio (retención de datos compartidos) es gestionable definiendo una política de retención antes de la implementación. Para RFC-02, se consideró que acotar el alcance de RNF-02 al entorno de producción es una aclaración de bajo riesgo que no reduce ninguna garantía real del sistema, solo evita un criterio de verificación inaplicable al MVP. Para RFC-03, el CCB determinó que formalizar el canal de respaldo in-app en RF-10 no introduce trabajo nuevo relevante, ya que la arquitectura de respaldo ya estaba contemplada en el entorno operativo; se trata de una corrección de consistencia entre RD-08 y RF-10.

RFC	Decisión	Acción y responsable	Plazo
RFC-01	Aprobado con condiciones	Incorporar RF-26 y definir política de retención de datos compartidos (Winston Cedeño)	09/08/2026
RFC-02	Aprobado	Actualizar redacción de RNF-02 (Andy Mendoza)	09/08/2026
RFC-03	Aprobado	Actualizar RF-10 y su criterio de verificación (Mayra Córdova)	09/08/2026

Tabla 10: Decisiones del CCB por RFC.

Las tres solicitudes fueron aprobadas por unanimidad; RFC-01 se aprueba con la condición de definir explícitamente la política de retención de datos antes de su implementación. El CCB acuerda que, una vez incorporados los tres cambios, se declare la nueva línea base ERS_SRS_2A_v1.1.

5.5. Versión resultante del ERS

Con la aprobación de las tres RFC, el archivo del ERS transita de v1.0 a v1.1 dentro del repositorio, correspondiendo internamente al salto de versión **3.0** → **3.1** en el historial de revisiones del propio documento (Entrega 3/2A ya vigente). Esta nueva versión incorpora: (i) las correcciones de los 18 defectos documentadas en la Sección 4; (ii) el nuevo RF-26 aprobado por RFC-01; (iii) la redacción acotada de RNF-02 aprobada por RFC-02; y (iv) la formalización del canal de respaldo en RF-10 aprobada por RFC-03.

La coherencia de versión se mantiene entre los siguientes artefactos: la portada y el historial de revisiones del propio documento `ERS_SRS_2A_v1.1` (que registra internamente el salto de 3.0 a 3.1), el archivo `CHANGELOG.md` del repositorio (entrada [1.1]) y el tag anotado `baseline-v1.1` publicado en el repositorio Git, conforme se detalla en la Sección 7.

6. Trazabilidad en herramienta CASE

6.1. Estructura del tablero

La trazabilidad de requisitos se gestionó en **Jira** (proyecto `MundiPets-PE4`, clave `KAN`), en modalidad Kanban, con acceso otorgado a los tres integrantes del equipo y al docente. El tablero se organizó en cinco columnas: **Backlog**, **Análisis**, **Aprobado**, **Desarrollo** y **Hecho**.

El backlog se estructuró en tres niveles jerárquicos:

- **5 Epics**, uno por cada área funcional del sistema: E1 — Gestión de mascotas y perfiles, E2 — Adopción y solicitudes, E3 — Historial médico y veterinaria, E4 — Cruza y compatibilidad, E5 — Cuenta, privacidad y comunicación.
- **26 Stories**, una por cada requisito funcional definido en el ERS (RF-01 a RF-26), superando el mínimo de 15 exigido.
- **10 Sub-tasks**, correspondientes a los criterios de aceptación de 5 historias de usuario (2 sub-tasks por historia): Registrar mascota (CA-01), Gestionar historial médico (CA-02), Consultar perfil completo (CA-03), Gestionar solicitudes de adopción (CA-04) y Evaluar compatibilidad para cruce (CA-05).

Al cierre de esta práctica, el tablero registraba 41 actividades totales (5 Epics, 26 Stories y 10 Sub-tasks), distribuidas en los cinco estados: 29 en Backlog, 7 en Análisis, 2 en Aprobado, 2 en Desarrollo y 1 en Hecho, conforme se detalla en la Sección 6.4.

6.2. Campos personalizados

Se configuraron dos campos personalizados a nivel de Story, aplicados a las 26 historias del backlog:

- **Prioridad MoSCoW**: lista desplegable con los valores *Must*, *Should*, *Could* y *Won't*, poblada directamente desde la prioridad ya definida para cada RF en el ERS.
- **Fuente del requisito**: campo de texto con el identificador de evidencia (Origen, ID de evidencia) que originó cada RF, por ejemplo `EV-02`, `EV-03`, `EV-05`, `EV-06`, `EV-07`, `EV-08` para RF-01.

Ambos campos quedaron completos en las 26 Stories (26/26), permitiendo tanto la trazabilidad hacia atrás (Fuente del requisito) como la priorización visible directamente desde el tablero (Prioridad MoSCoW), sin necesidad de consultar el ERS para conocer el origen o la importancia de cada historia.

6.3. Matriz de trazabilidad

La Tabla 11 presenta la trazabilidad de las 26 Stories cargadas en el tablero CASE (Jira), en el formato Stakeholder → RF → CU → Clase/Módulo → Prueba. El stakeholder corresponde al participante que originó cada requisito (columna *Fuente del requisito* del tablero), la clase o módulo se obtiene del diagrama de clases refinado del ERS (Sección 5.6, *Diagrama de clases refinado*), y el identificador de prueba (CP-XX) corresponde al caso de prueba asociado a cada requisito funcional.

Tabla 11: Matriz de trazabilidad extendida: Stakeholder → RF → CU → Clase/Módulo → Prueba.

RF	Stakeholder	CU	Clase/Módulo	Prueba
RF-01	PROP-02 (propietaria)	CU-01	Mascota	CP-01
RF-03	PROP-02 (propietaria)	CU-10	Mascota, HistorialMedico	CP-03
RF-04	Usuarios encuestados	CU-11	Mascota (búsqueda)	CP-04
RF-13	PROP-01 (propietario)	CU-04	Publicacion	CP-13
RF-17	PROP-01 (propietario)	CU-04	Publicacion, Módulo de IA	CP-17
RF-21	VET-05 (veterinario)	CU-01	Microchip	CP-21
RF-25	PROP-07 (propietaria)	—	AvisoExtraviado	CP-25
RF-05	PROP-01 (propietario)	CU-04	SolicitudAdopcion	CP-05
RF-15	PROP-01 (propietario)	CU-07	SolicitudAdopcion, SolicitudCruza	CP-15
RF-18	PROP-06 (propietaria)	CU-04	SolicitudAdopcion	CP-18
RF-20	PROP-08 (propietario)	—	SolicitudAdopcion (extensión)	CP-20
RF-22	VET-05 (veterinario)	CU-07	SolicitudAdopcion (seguimiento)	CP-22
RF-23	PROP-04 (propietario)	CU-06	AlertaRiesgo	CP-23
RF-02	PROP-01 (propietario)	CU-02	HistorialMedico	CP-02
RF-09	VET-01 (veterinaria)	CU-09	HistorialMedico, RD-03	CP-09
RF-14	PROP-02 (propietaria)	CU-03	HistorialMedico (vacunas)	CP-14
RF-24	VET-04 (veterinaria)	CU-03	HistorialMedico (vacunas)	CP-24
RF-06	PROP-01 (propietario)	CU-06	RegistroCompatibilidad, Mó- dulo de IA	CP-06
RF-08	PROP-01 (propietario)	CU-05	SolicitudCruza	CP-08
RF-16	PROP-01 (propietario)	CU-13	AntecedenteGenetico	CP-16
RF-19	PROP-06 (propietaria)	CU-06	CertificadoVeterinario	CP-19
RF-11	PROP-02 (propietaria)	CU-12	Usuario (privacidad)	CP-11
RF-12	PROP-01 (propietario)	CU-09	Usuario	CP-12
RF-07	PROP-02 (propietaria)	CU-08	Mensaje	CP-07
RF-10	PROP-02 (propietaria)	—	HistorialMedico (recordato- rios)	CP-10
RF-26	RL-04 (LOPDP, Art. 15)	—	Usuario (eliminación)	CP-26

Requisitos huérfanos. Cuatro de los 26 requisitos trazados (RF-25, RF-20, RF-10 y RF-26) no cuentan con un Caso de Uso (CU) formalmente documentado en el ERS, por lo que se reportan explícitamente como huérfanos hacia adelante en esta dimensión: RF-25 (Publicar aviso de mascota extraviada) y RF-20 (Coordinar encuentros de socialización) son funcionalidades de prioridad *Could* incorporadas en la última entrega sin un CU dedicado; RF-10 (Gestionar recordatorios) se apoya en el flujo de RF-14 sin CU propio; y RF-26 (Eliminar cuenta y datos personales) es un requisito nuevo incorporado por RFC-01 en esta misma práctica, pendiente de un CU dedicado en una futura iteración del modelado UML. En los cuatro casos, la trazabilidad hacia atrás (stakeholder/fuente) y hacia la clase/módulo y prueba correspondientes sí está completa.

6.4. Evidencias del tablero

La Figura 2 muestra el tablero Kanban completo, con las cinco columnas de trabajo y las tarjetas distribuidas entre los distintos estados al cierre de esta práctica.

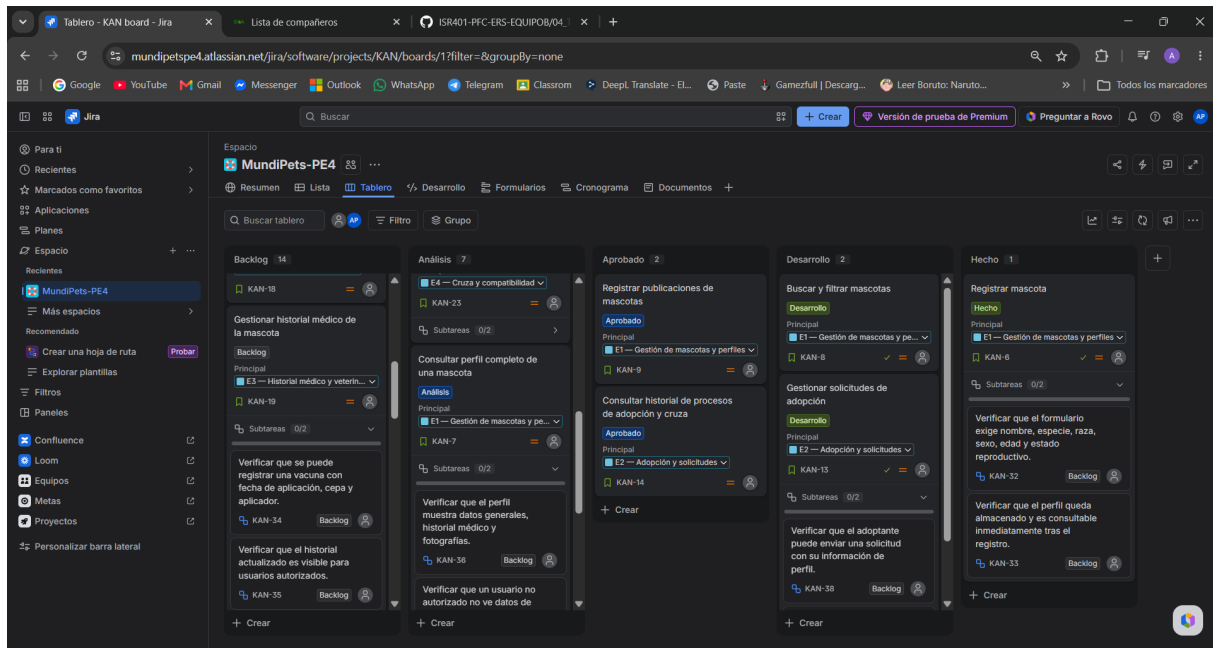


Figura 2: Tablero Kanban completo del proyecto MundiPets-PE4 en Jira.

La Figura 3 muestra el panel de estadísticas del proyecto (pestaña “Resumen”), con el desglose de 41 actividades por estado, tipo de trabajo y prioridad.

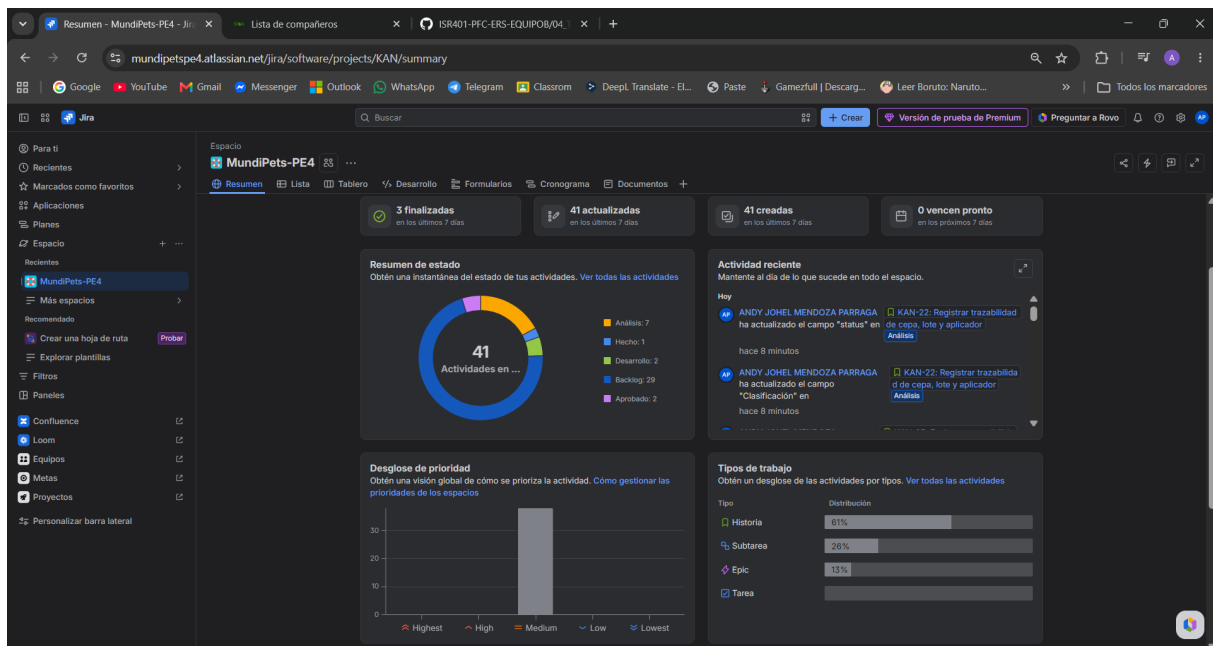


Figura 3: Panel de estadísticas del proyecto MundiPets-PE4.

Las Figuras 4, 5 y 6 muestran el detalle de tres issues representativos, evidenciando los campos personalizados completos (Prioridad MoSCoW, Fuente del requisito), la descripción con el RF y el CU enlazados, y las sub-tasks asociadas.

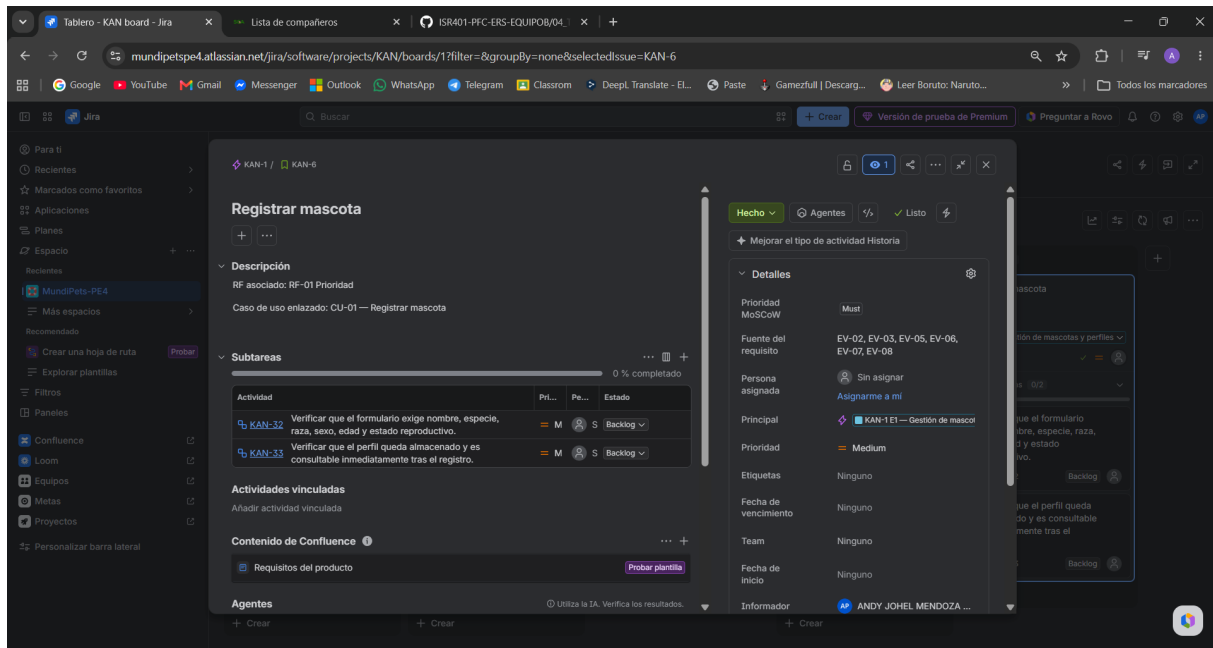


Figura 4: Detalle de la Story “Registrar mascota” (RF-01), con sus dos sub-tasks (CA-01) y campos personalizados.

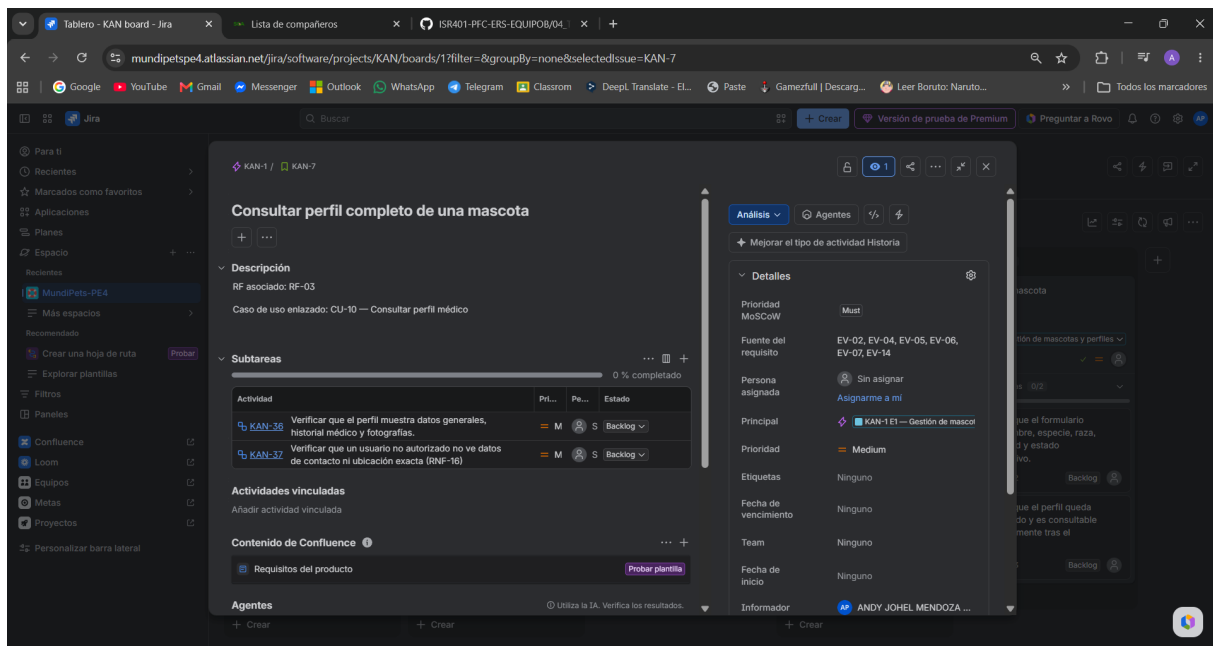


Figura 5: Detalle de la Story “Consultar perfil completo de una mascota” (RF-03), con sus dos sub-tasks (CA-03) y campos personalizados.

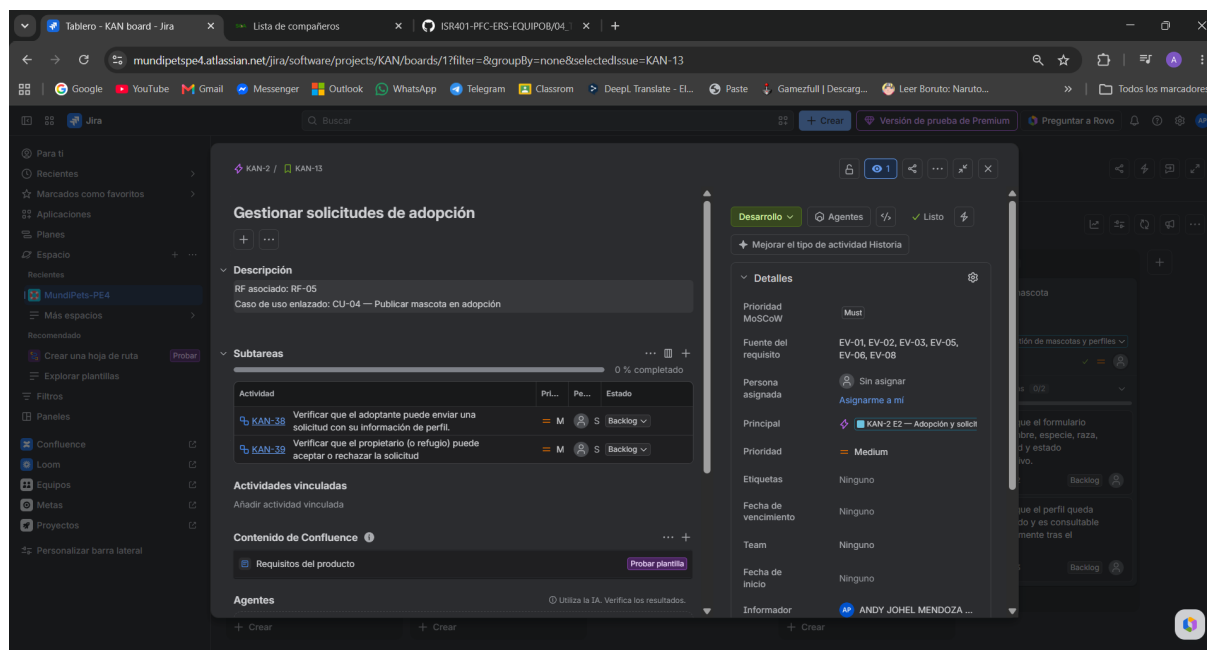


Figura 6: Detalle de la Story “Gestionar solicitudes de adopción” (RF-05), con sus dos sub-tasks (CA-04) y campos personalizados.

Adicionalmente, el backlog completo (41 issues: 5 Epics, 26 Stories y 10 Sub-tasks) fue exportado en formato CSV con todos los campos, y se encuentra disponible en el repositorio como 04_Trazabilidad/backlog_export.csv (Anexo D), junto con la matriz de trazabilidad en formato .xlsx (04_Trazabilidad/matriz_trazabilidad.xlsx).

7. Línea base del ERS con Git

7.1. Repositorio y estructura

El repositorio del equipo, ISR401-PFC-ERS-EQUIPOB, es público y está alojado en GitHub (<https://github.com/andymendozap03/ISR401-PFC-ERS-EQUIPOB>). Contiene un README.md con: el sistema del PFC (MundiPets), los tres integrantes con su rol, la estructura completa de carpetas (01_ERS a 07_Borradores) y las instrucciones de compilación del informe y del ERS — distribución LaTeX requerida, orden de comandos (pdflatex → biber → pdflatex ×2), archivo principal de cada documento y dependencias de paquetes.

La estructura del repositorio separa claramente los artefactos de cada paso de la práctica: 01_ERS/ contiene el ERS versionado (v1.0 y v1.1, con sus fuentes LaTeX y figuras); 02_Inspeccion/ y 03_CCB/ contienen la evidencia firmada de la inspección Fagan y del CCB; 04_Trazabilidad/ contiene el backlog exportado y la matriz de trazabilidad; 05_Informe/ contiene el informe de la PE4; y 06_Evidencias/ contiene las capturas y fotografías de las sesiones de trabajo.

7.2. Commits semánticos

El equipo registró un número creciente de commits distribuidos a lo largo de toda la práctica, con autoría real de los tres integrantes (andymendozap03, mcordovac2, WinstonCD), superando ampliamente el mínimo de 8 commits exigido; el desglose exacto por integrante puede verificarse en la vista de commits de GitHub (Figuras 9 y 10). Los mensajes siguen una convención semántica consistente, con prefijos como docs(ers), docs(pe4), docs(inspeccion), docs(ccb), docs(trazabilidad), docs(changelog) y fix(inspeccion) según el tipo de cambio. Un ejemplo representativo:

```
docs(ers): v1.1 - acotar RNF-02, agregar criterio de verificacion a las
RD y corregir matriz de trazabilidad (40->59 filas)
```

La mayoría de los commits aparecen marcados como *Verified* en GitHub, confirmando que fueron firmados con las credenciales reales de cada integrante.

7.3. Tag anotado de línea base

Una vez incorporados los cambios aprobados por el CCB (RFC-01, RFC-02 y RFC-03) y corregidos los 18 defectos de la inspección, se declaró la nueva línea base del ERS mediante un tag anotado:

```
git tag -a baseline-v1.1 -m "Baseline aprobada por CCB"
git push origin baseline-v1.1
```

El tag `baseline-v1.1` quedó publicado en el repositorio remoto, apuntando al commit `ee63ed3` (`docs(changelog): completar entrada [1.1] con el detalle de cambios aplicados al ERS`), que es el commit que consolida el cierre de la versión.

7.4. CHANGELOG.md

El archivo `CHANGELOG.md` del repositorio documenta ambas versiones del ERS en el formato `[Versión] - [Fecha] / Añadido / Cambiado / Eliminado`. La entrada `[1.0]` (14/06/2026) registra la versión base importada desde la Entrega 2A. La entrada `[1.1]` (09/08/2026) detalla individualmente cada uno de los cambios aprobados por RFC (RF-26, RNF-02 acotado, canal de respaldo en RF-10) y cada corrección de defecto aplicada durante la inspección (RF-01, RF-02, RF-05, RF-12, RF-18, RF-24, los RNF de muestra, RNF-11, RNF-16, la definición de titular, y la corrección de la matriz de trazabilidad de 40 a 59 filas), incluyendo una nota que aclara la relación entre el versionado del archivo (`v1.0` → `v1.1`) y la versión interna del propio documento (`3.0` → `3.1`).

7.5. Historial de commits (evidencia)

La Figura 7 muestra la salida del comando `git log --oneline --graph --decorate`, evidenciando el historial lineal completo del repositorio, con el tag `baseline-v1.1` anotado sobre el commit más reciente (`HEAD` → `main`). La Figura 8 confirma, mediante `git tag -n`, que el tag quedó correctamente anotado con su mensaje descriptivo.

```
Windows PowerShell
To https://github.com/andymendoza03/ISR401-PFC-ERS-EQUIPOB.git
32abb6d..ee63ed3 main -> main
PS C:\Users\Andy Mendoza\Music\ISR401-PFC-ERS-EQUIPOB> git pull
Already up to date.
PS C:\Users\Andy Mendoza\Music\ISR401-PFC-ERS-EQUIPOB> git tag -a baseline-v1.1 -m "Baseline aprobada por CCB"
PS C:\Users\Andy Mendoza\Music\ISR401-PFC-ERS-EQUIPOB> git push origin baseline-v1.1
Enumerating objects: 1, done.
Counting objects: 100% (1/1), 180 bytes | 180.00 KiB/s, done.
Writing objects: 100% (1/1), 180 bytes | 180.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/andymendoza03/ISR401-PFC-ERS-EQUIPOB.git
 * [new tag]         baseline-v1.1 -> baseline-v1.1
PS C:\Users\Andy Mendoza\Music\ISR401-PFC-ERS-EQUIPOB> git log --oneline --graph --decorate
* ee63ed3 (HEAD -> main, tag: baseline-v1.1, origin/main, origin/HEAD) docs(changelog): completar entrada [1.1] con el detalle de cambios aplicados al ERS
* 32abb6d docs(ers): v1.1 - acotar RNF-02, agregar criterio de verificación a las RD y corregir matriz de trazabilidad (40->59 filas)
* 4b01acc docs(ers): formalizar canal de respaldo en RF-10 (RFC-03) y corregir defectos de Inspector 2
* 863505a docs(ers): incorporar RF-26 (RFC-01) y corregir defectos detectados por Inspector
* 8fef014 docs(ccb): agregar formularios RFC y acta del CCB firmados
* 2afa5c9 docs(pe4): completar sección 5 - RFC-01, RFC-02, RFC-03 y acta del CCB
* eac10de docs(pe4): completar sección 4 - correcciones aplicadas a defectos críticos y mayores
* 0b0a233 docs(inspection): completar sección 3 del informe (metricas) y agregar metricas.xlsx
* c560d76 docs(inspection): agregar evidencia fotografica de la reunion
* c673abf fix(inspection): corregir referencia de sección en Anexo A (5.3 -> 5.5, matriz de trazabilidad)
* 9935503 docs(inspection): consolidar Anexo B - 18 defectos detectados en reunion
* c23052b docs(inspection): correccion planificación de roles firmada
* 75762b1 docs(inspection): correccion Anexo A - preparacion individual (Inspector 2)
* 2ea5d45 docs(inspection): completar Anexo A - preparacion individual (Inspector 1)
* 194f28c docs(inspection): completar Anexo A - preparacion individual (Inspector 2)
* 9094851 docs(pe4): completar fundamento teorico 2.1-2.4
* da06241 docs(borrador): agregar borrador de redaccion de fundamento teorico 2.1-2.4
* cb951a7 docs(inspection): planificación de roles firmada y Anexo A (Moderador/Lector)
* da2e555 docs(pe4): integrar fundamento teorico 3.4 (herramientas CASE) y 3.5 (IR ágil)
* b390ae1 docs(borrador): agregar borrador de redaccion de fundamento teorico 2.4-2.5
* a710a9e docs(pe4): estructura completa del informe con objetivos redactados
* b5d962c docs(ers): v1.0 - estructura inicial del repositorio e importación del ERS de la Entrega 2A
* f036ae8 Initial commit
PS C:\Users\Andy Mendoza\Music\ISR401-PFC-ERS-EQUIPOB> git tag -n
baseline-v1.1 Baseline aprobada por CCB
PS C:\Users\Andy Mendoza\Music\ISR401-PFC-ERS-EQUIPOB> |
```

Figura 7: Salida de `git log --oneline --graph --decorate` y `git tag -n`, mostrando el historial completo de commits y el tag `baseline-v1.1`.

```

Already up to date.
PS C:\Users\Andy Mendoza\Music\ISR401-PFC-ERS-EQUIPOB> git tag -a baseline-v1.1 -m "Baseline aprobada por CCB"
PS C:\Users\Andy Mendoza\Music\ISR401-PFC-ERS-EQUIPOB> git push origin baseline-v1.1
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 180 bytes | 180.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/andymendozap03/ISR401-PFC-ERS-EQUIPOB.git
 * [new tag]         baseline-v1.1 -> baseline-v1.1
PS C:\Users\Andy Mendoza\Music\ISR401-PFC-ERS-EQUIPOB> git log --oneline --graph --decorate
* ee63ed3 (HEAD -> main, tag: baseline-v1.1, origin/main, origin/HEAD) docs(changelog): completar entrada [1.1] con el detalle de cambios aplicados al ERS
* 32abb6d docs(ers): v1.1 - acotar RNF-02, agregar criterio de verificación a las RD y corregir matriz de trazabilidad (40->59 filas)
* 4b01acc docs(ers): formalizar canal de respaldo en RF-10 (RFC-03) y corregir defectos de Inspector 2
* 863505a docs(ers): incorporar RF-26 (RFC-01) y corregir defectos detectados por Inspector
* 8fef014 docs(ccb): agregar formularios RFC y acta del CCB firmados
* 2efa5c9 docs(pe4): completar sección 5 - RFC-01, RFC-02, RFC-03 y acta del CCB
* eac10de docs(pe4): completar sección 4 - correcciones aplicadas a defectos críticos y mayores
* 0b0a233 docs(inspection): completar sección 3 del informe (metricas) y agregar metricas.xlsx
* c560d76 docs(inspection): agregar evidencia fotografica de la reunion
* c673abf fix(inspection): corregir referencia de sección en Anexo A (5.3 -> 5.5, matriz de trazabilidad)
* 9935503 docs(inspection): consolidar Anexo B - 18 defectos detectados en reunion
* c23052b docs(inspection): corrección planificación de roles firmada
* 75762b1 docs(inspection): corrección Anexo A - preparación individual (Inspector 2)
* 2ea5df5 docs(inspection): completar Anexo A - preparación individual (Inspector 1)
* 194f28c docs(inspection): completar Anexo A - preparación individual (Inspector 2)
* 9494851 docs(pe4): completar fundamento teórico 2.1-2.4
* da06241 docs(borrador): agregar borrador de redacción de fundamento teórico 2.1-2.4
* cb951a7 docs(inspection): planificación de roles firmada y Anexo A (Moderador/Lector)
* da2a555 docs(pe4): integrar fundamento teórico 3.4 (herramientas CASE) y 3.5 (IR ágil)
* b398ae1 docs(borrador): agregar borrador de redacción de fundamento teórico 2.4-2.5
* a710a9e docs(pe4): estructura completa del informe con objetivos redactados
* b5d962c docs(ers): v1.0 - estructura inicial del repositorio e importación del ERS de la Entrega 2A
* f036ae8 Initial commit
PS C:\Users\Andy Mendoza\Music\ISR401-PFC-ERS-EQUIPOB> |

```

Figura 8: Detalle ampliado del historial de commits, con autoría distribuida entre los tres integrantes del equipo.

Adicionalmente, las Figuras 9 y 10 muestran la vista de commits en GitHub, confirmando la autoría real de cada integrante (andymendozap03, mcordovac2, WinstonCD) y el estado *Verified* de la mayoría de los commits.

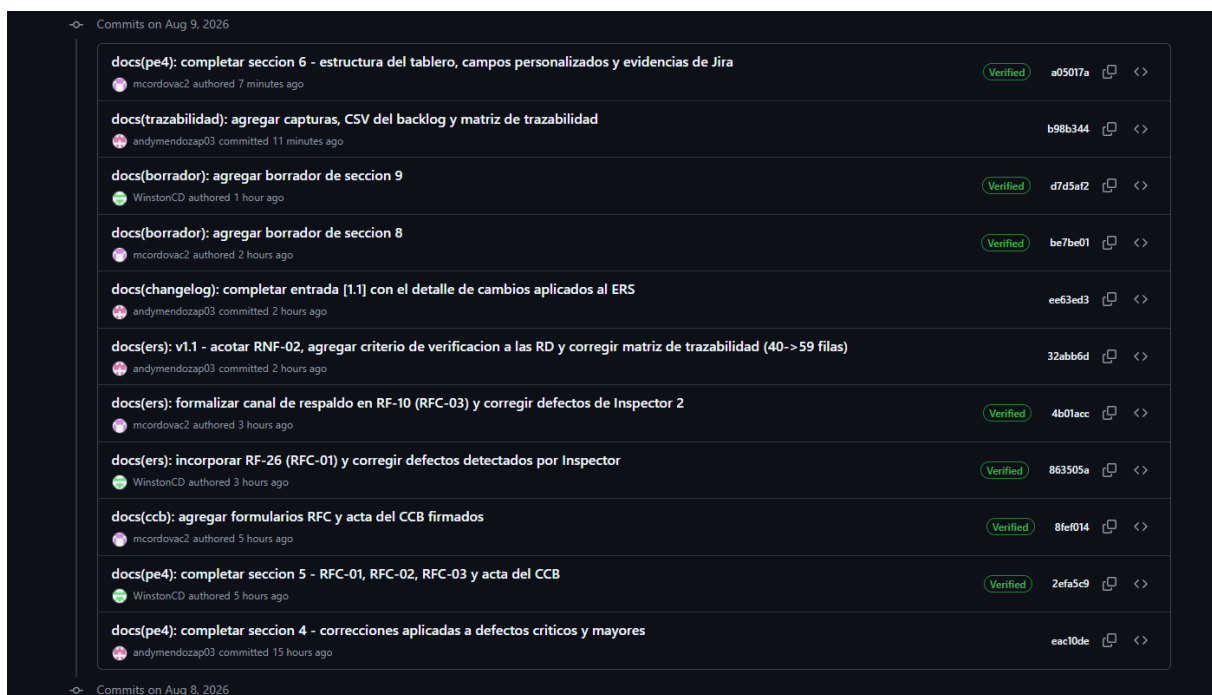


Figura 9: Vista de GitHub con los commits más recientes del repositorio, mostrando autoría distribuida.

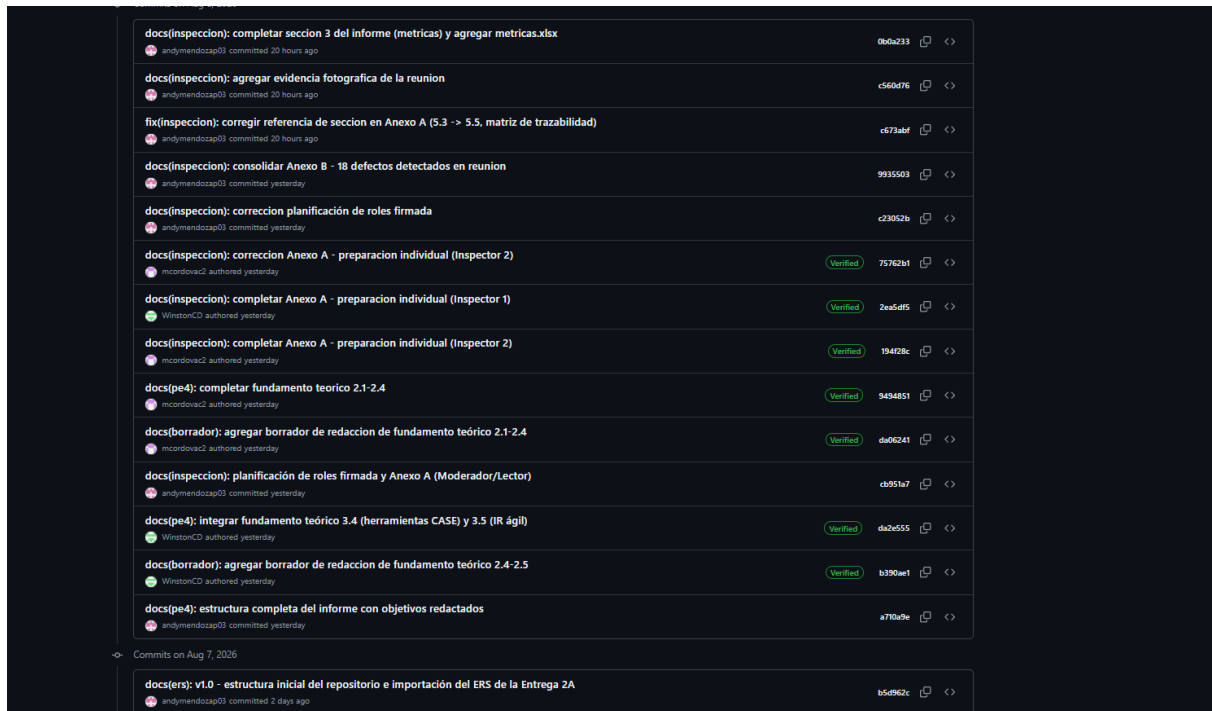


Figura 10: Continuación del historial de commits en GitHub.

8. Comparativa de herramientas CASE

La selección de una herramienta CASE para ingeniería de requisitos debe considerar tanto sus capacidades funcionales como las condiciones específicas del proyecto y del equipo que la utilizará. Tras analizar 56 herramientas de ingeniería de requisitos, Ozkaya et al. agrupan sus características en dimensiones como gestión de proyectos, especificación, colaboración, personalización, interoperabilidad, metodología y soporte al usuario, y señalan que ninguna herramienta proporciona un nivel uniforme de soporte para todas ellas [17]. De forma complementaria, Nadeem et al. comparan herramientas de obtención y gestión de requisitos considerando características como gestión jerárquica, verificación, análisis de calidad, versionado, gestión de pruebas y generación de reportes [18]. La selección tampoco debe reducirse al número de funcionalidades disponibles: Bjarnason et al. sostienen que la evaluación de software debe considerar criterios técnicos y organizacionales en función del contexto donde será adoptado [19].

A partir de estos principios y de la documentación oficial disponible, se comparan Jira, IBM Engineering Requirements Management DOORS Next, Jama Connect y ReqView mediante seis criterios relevantes para el PFC MundiPets.

Tabla 12: Comparativa de herramientas CASE para gestión de requisitos.

Criterio	Jira	IBM DOORS Next	Jama Connect	ReqView
Funcionalidades de gestión de requisitos	Media-Alta. Flexible mediante issues, historias, épicas y workflows, especialmente en proyectos ágiles; no nació como herramienta especializada en IR	Muy alta. Diseñado específicamente para requisitos de sistemas y software, con gestión de cambios, revisiones e integración con el ciclo de vida [27]	Muy alta. Orientado a requisitos, pruebas, riesgos, workflows y revisiones; sólido en procesos formales de validación [28]	Alta. Permite documentar requisitos, atributos, riesgos, verificaciones y validaciones, con una aproximación más ligera que DOORS o Jama [29]

Criterio	Jira	IBM Next	DOORS	Jama Connect	ReqView
Trazabilidad	Media-Alta. Enlaces entre issues permiten relaciones, aunque una trazabilidad formal extremo a extremo requiere estructurar cuidadosamente tipos y enlaces	Muy alta. Es una capacidad central; relaciona requisitos con otros artefactos mediante el ecosistema IBM ELM [27]		Muy alta. Trazabilidad hacia adelante y hacia atrás, análisis de impacto e informes, apropiada para proyectos complejos o regulados [30]	Muy alta. Permite crear relaciones, visualizar matrices de trazabilidad, analizar impactos y gestionar enlaces sospechosos [29]
Colaboración	Muy alta. Comentarios, asignaciones y workflows facilitan la colaboración continua, particularmente apropiado para equipos ágiles	Alta. Incorpora revisiones, edición visual, generación de documentos e historial de auditoría [27]		Muy alta. Revisiones, comentarios y distintos tipos de participantes favorecen la colaboración entre ingeniería, producto y calidad [28]	Alta. Permite comentarios y trabajo compartido mediante unidades de red, almacenamiento cloud, Git o SVN [29]
Integración con Git/VCS	Muy alta para desarrollo de software; se adapta bien a ecosistemas donde requisitos y tareas se relacionan con repositorios de código	Media-Alta. Su fortaleza está en la integración con el ecosistema IBM ELM; un flujo centrado en Git puede resultar más complejo		Alta. Mantiene trazabilidad con otras herramientas del ciclo de desarrollo, aunque algunas integraciones dependen de infraestructura adicional	Muy alta. Git y SVN forman parte directa de su estrategia de versionado, control de historial y gestión de baselines [29]
Costo	Favorable para equipos pequeños; dispone de opciones de entrada, funciones avanzadas dependen del plan	Alto/empresarial. Dirigido principalmente a organizaciones que necesitan gestión formal y escalable; puede resultar sobredimensionado para un PFC		Alto/empresarial. Licenciamiento según tipos de usuario; no presenta precio único público [28]	Favorable. Planes Free, Pro, Team y Enterprise; la edición gratuita permite evaluar la herramienta antes de adquirir un plan superior [29]
Curva de aprendizaje	Baja-Media. Organización mediante issues y tableros es accesible, aunque un proceso formal de requisitos exige configuración previa	Alta. Su amplitud funcional proporciona gran control, pero requiere mayor configuración y capacitación		Media-Alta. Sus capacidades especializadas requieren familiarización inicial, justificable en proyectos complejos	Baja-Media. Enfoque documental y estructura relativamente sencilla permiten comenzar con menor configuración [29]

Las valoraciones “baja”, “media”, “alta” y “muy alta” son juicios comparativos, no clasificaciones declaradas por los fabricantes: se derivan de las capacidades documentadas y de los criterios de selección propuestos por la literatura [17, 19].

Recomendación para el PFC MundiPets

Para el PFC MundiPets se recomienda **ReqView** como la alternativa que presenta el equilibrio más favorable entre gestión formal de requisitos, trazabilidad, control de versiones, costo y curva de aprendizaje. Aunque IBM DOORS Next y Jama Connect ofrecen capacidades más amplias para organizaciones y proyectos de ingeniería complejos, esa amplitud también implica un nivel de infraestructura, licenciamiento y aprendizaje que puede resultar innecesario para el alcance de un proyecto académico. Jira, por su parte, destaca en colaboración y en su cercanía con el flujo de desarrollo ágil, pero requiere mayor adaptación para funcionar como repositorio formal de requisitos.

ReqView resulta particularmente adecuado porque integra documentación estructurada de requisitos, atributos, trazabilidad, verificación y validación, análisis de impacto y control de cambios, y además permite utilizar Git o SVN para versionar proyectos y gestionar baselines. Estas características se relacionan directamente con los mecanismos desarrollados en el fundamento teórico — ERS, línea base, trazabilidad y gestión de cambios — y permiten aplicarlos en el PFC sin incorporar la complejidad de una plataforma empresarial de mayor escala. Esta decisión responde al criterio planteado: la herramienta no debería seleccionarse simplemente por disponer del mayor número de funcionalidades, sino por su adecuación al contexto y necesidades concretas del proyecto [19].

9. IR tradicional frente a IR ágil

La Ingeniería de Requisitos (IR) tradicional y la IR ágil presentan diferencias en la forma de documentar, gestionar y evolucionar los requisitos. Estas diferencias no implican que un enfoque sea mejor que otro, sino que su conveniencia depende de las características y restricciones del proyecto.

Tabla 13: IR tradicional frente a IR ágil en seis dimensiones.

Dimensión	IR tradicional (documental)	IR ágil	Contexto de conveniencia
Documentación	Se apoya en una especificación formal de requisitos, línea base y mecanismos de trazabilidad verificable. El énfasis está en mantener una referencia documental controlada durante el ciclo de vida [3]	Prioriza artefactos ligeros y vivos, como el product backlog, las historias de usuario y los criterios de aceptación. La documentación se construye y refina de manera incremental [23]	Conviene cuando se requiere evidencia formal, control documental o trazabilidad verificable. El enfoque ágil conviene cuando los requisitos evolucionan continuamente
Gestión del cambio	Los cambios se gestionan de manera controlada sobre los requisitos y sus versiones, procurando conservar la trazabilidad y el impacto de las modificaciones [3]	Los cambios se incorporan mediante el refinamiento y la repriorización continua del backlog, permitiendo adaptar las necesidades durante las iteraciones [23]	El tradicional es adecuado cuando los cambios tienen implicaciones contractuales, regulatorias o críticas. El ágil es conveniente cuando existe alta variabilidad de requisitos
Participación de stakeholders	La participación de los interesados se integra en actividades de elicitación y validación, buscando comprobar que los requisitos representan sus necesidades y los objetivos del sistema [3]	La participación es continua y cercana al desarrollo. Las historias de usuario representan necesidades desde la perspectiva del usuario y se refinan con retroalimentación frecuente [23]	El tradicional resulta conveniente cuando existen aprobaciones y revisiones formales. El ágil es apropiado cuando el cliente o los usuarios pueden participar de manera frecuente

Dimensión	IR tradicional (documental)	IR ágil	Contexto de conveniencia
Herramientas	Puede utilizar herramientas CASE para especificación estructurada, versionado, trazabilidad y gestión de cambios, manteniendo relaciones controladas entre los artefactos [3]	Utiliza herramientas colaborativas para gestionar backlog, historias de usuario y trazabilidad ligera. También pueden vincularse commits, historias y casos de prueba [23]	Las herramientas con control documental son apropiadas cuando se necesita auditoría y trazabilidad formal. Las colaborativas son convenientes para equipos iterativos
Velocidad	Requiere mayor esfuerzo inicial para especificar, revisar y establecer una referencia controlada de los requisitos [3]	Favorece ciclos cortos e incrementales y permite responder con mayor rapidez a cambios mediante el refinamiento continuo del backlog [23]	El enfoque ágil conviene especialmente con plazos cortos y necesidad de adaptación rápida. El tradicional es apropiado cuando la estabilidad y previsibilidad son prioritarias
Escalabilidad	La documentación formal, las versiones y la trazabilidad permiten mantener control en proyectos complejos, aunque aumenta el esfuerzo de mantenimiento documental [3]	Puede escalar mediante backlog, herramientas y trazabilidad ligera; sin embargo, la reducción de documentación puede afectar especialmente la trazabilidad de requisitos no funcionales [26]	En proyectos grandes, regulados o con alta exigencia de trazabilidad conviene reforzar el enfoque documental. En proyectos iterativos puede utilizarse el enfoque ágil con mecanismos complementarios

Discusión

La elección entre IR tradicional e IR ágil debe entenderse como una decisión condicionada por el contexto del proyecto y no como una comparación para determinar cuál enfoque es mejor. Los proyectos con alta variabilidad de requisitos, plazos cortos y participación constante del cliente se benefician de prácticas ágiles de gestión de requisitos [23]. En cambio, los proyectos regulados, de misión crítica o con requisitos contractuales estrictos requieren conservar un nivel de documentación formal y trazabilidad verificable, especialmente cuando están sujetos a normas como ISO/IEC/IEEE 29148:2018 [3, 23]. Asimismo, en contextos ágiles se debe prestar especial atención a los requisitos no funcionales, ya que su documentación puede ser descuidada y requerir mecanismos complementarios de trazabilidad [26]. Por ello, cuando las condiciones del proyecto lo exigen, también puede resultar conveniente un esquema híbrido que conserve una especificación de alto nivel y una línea base formal, mientras administra el detalle operativo mediante backlog e historias de usuario [23].

10. Conclusiones críticas

- 1. La densidad de defectos detectada (0.36 por página) revela que los errores de un ERS que ya pasó por dos entregas previas no son de redacción individual, sino de coherencia entre secciones.** De los 18 defectos consolidados en la inspección Fagan, más del 60 % correspondieron a completitud y consistencia (11 de 18), y no a ambigüedad o factibilidad. El caso más ilustrativo fue la propia matriz de trazabilidad del ERS, que declaraba simultáneamente “40 filas” y “50 elementos” en el mismo párrafo: un defecto que ninguna revisión superficial de un único requisito habría detectado, y que solo emergió al leer el documento de forma sistemática y completa, como exige el método Fagan.
- 2. Que los tres integrantes detectaran exactamente 6 defectos cada uno no garantiza, por sí solo, independencia real en la revisión.** La tasa de detección perfectamente equilibrada (33.3 % cada uno) podría interpretarse como evidencia de un proceso saludable, pero al revisar el contenido, cada inspector concentró sus hallazgos en secciones distintas del ERS (Winston en requisitos funcionales y no funcionales, Mayra en historias de usuario y entorno operativo, Andy en restricciones de diseño y la matriz de trazabilidad). Esto sugiere que el equilibrio numérico reflejó más una división implícita del trabajo que una verificación cruzada exhaustiva sobre las mismas secciones, un matiz que las métricas agregadas por sí solas no muestran.
- 3. El CCB simulado evidenció que no todo cambio derivado de una inspección requiere el mismo nivel de análisis de impacto.** De las tres RFC tramitadas, solo RFC-01 (incorporación de RF-26) fue aprobada *con condiciones*, al implicar una política de retención de datos aún no definida; RFC-02 y RFC-03 se aprobaron sin reservas, por tratarse de ajustes de redacción y consistencia sin riesgo estructural. Esto confirma en la práctica que el criterio de *riesgo* de una RFC, y no solo su origen (defecto crítico vs. mayor), es lo que determina el nivel de escrutinio real que amerita en el comité.
- 4. La trazabilidad construida en Jira expuso 4 requisitos huérfanos (RF-25, RF-20, RF-10 y RF-26) que el ERS no había señalado explícitamente como tales.** Al enlazar sistemáticamente las 26 Stories del backlog con sus Casos de Uso correspondientes, se hizo evidente que ciertos requisitos de prioridad *Could* y el requisito nuevo aprobado por RFC-01 carecían de un CU dedicado en el modelado UML del ERS. Este hallazgo no habría surgido de la sola lectura del documento, sino del ejercicio de forzar cada requisito a través de una cadena de trazabilidad completa en la herramienta CASE.
- 5. La comparativa de herramientas CASE mostró que la herramienta más adecuada para gestionar el ERS a largo plazo no es necesariamente la más práctica para el ejercicio de trazabilidad de esta unidad.** El equipo usó Jira para construir el backlog exigido por la guía, por su bajo costo de adopción y su fortaleza para trabajo ágil; sin embargo, el análisis comparativo de seis criterios llevó a recomendar ReqView para la gestión formal y sostenida del ERS del PFC, por su equilibrio entre documentación estructurada, trazabilidad, control de versiones y curva de aprendizaje. Esta disociación entre la herramienta usada y la recomendada evidencia que la elección de una herramienta CASE depende del horizonte temporal y el propósito específico de su uso, no de una preferencia única y universal del equipo.

11. Distribución del trabajo

La Tabla 14 resume el aporte de cada integrante, contrastable directamente con el historial de commits del repositorio (<https://github.com/andymendozap03/ISR401-PFC-ERS-EQUIPOB>).

Integrante	Aporte
Mendoza Párraga Andy Johel	Rol: Moderador y Lector (inspección Fagan), Presidente del CCB. Redactó y estructuró el informe completo; condujo la reunión de inspección y consolidó el Anexo B; calculó las métricas de inspección (Sección 3.4); documentó las correcciones aplicadas a los defectos críticos y mayores (Sección 4); aplicó al ERS la corrección de RNF-02 (RFC-02), el criterio de verificación de las 9 restricciones de diseño y la corrección de la matriz de trazabilidad (40→59 filas); cerró la gestión documental (CHANGELOG, tag <code>baseline-v1.1</code>) y redactó la Sección 7 (Línea base con Git).
Cedeño Ávila Winston Damian	Rol: Inspector 1 (inspección Fagan), Analista de requisitos (CCB). Redactó el fundamento teórico de herramientas CASE e IR ágil (secciones 2.4 y 2.5); realizó su preparación individual de la inspección (6 defectos); solicitó y sustentó la RFC-01 (incorporación de RF-26); aplicó al ERS los cambios correspondientes a RFC-01 y corrigió los defectos que él mismo detectó (RF-12, RF-24, RNF-11, RNF-16, entre otros); redactó la Sección 5 (RFC-01, RFC-02, RFC-03 y acta del CCB) y la Sección 9 (IR tradicional frente a IR ágil).
Córdova Carreño Mayra Lucila	Rol: Inspector 2 (inspección Fagan), Representante del cliente / Desarrolladora (CCB). Redactó el fundamento teórico de marco normativo, validación y gestión de requisitos (secciones 2.1 a 2.3); realizó su preparación individual de la inspección (6 defectos); solicitó y sustentó la RFC-03 (canal de respaldo en RF-10); aplicó al ERS los cambios correspondientes a RFC-03 y corrigió los defectos que ella misma detectó (RF-05, RF-02, RF-18, entre otros); construyó y documentó la estructura del tablero Jira (Sección 6); redactó la comparativa de herramientas CASE (Sección 8).

Tabla 14: Distribución del trabajo por integrante, PE4 (Unidad IV).

El repositorio registra un número creciente de commits distribuidos con autoría real entre los tres integrantes, superando ampliamente el mínimo de 8 commits exigido por la guía; el conteo exacto y actualizado puede verificarse directamente en el historial de `git log` del repositorio (Sección 6.4 y Figura 7). La distribución de roles se mantuvo consistente a lo largo de toda la práctica: los mismos roles asignados en la planificación de la inspección Fagan (Sección 3.1) se conservaron en el Change Control Board (Sección 5.4), reforzando la trazabilidad de responsabilidades individuales dentro del equipo.

12. Referencias

- [1] L. Montgomery, D. Fucci, A. Bouraffa, L. Scholz y W. Maalej, “Empirical research on requirements quality: a systematic mapping study,” *Requirements Engineering*, vol. 27, págs. 183-209, 2 jun. de 2022, ISSN: 0947-3602. DOI: 10.1007/s00766-021-00367-z
- [2] M. A. Umar y K. Lano, “Advances in automated support for requirements engineering: a systematic literature review,” *Requirements Engineering*, vol. 29, págs. 177-207, 2 jun. de 2024, ISSN: 0947-3602. DOI: 10.1007/s00766-023-00411-0
- [3] ISO/IEC/IEEE, “ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Life Cycle Processes – Requirements Engineering,” International Organization for Standardization, inf. téc., 2018.
- [4] ISO/IEC/IEEE, *15288:2023 — System life cycle processes*, 2023.
- [5] ISO/IEC/IEEE, “ISO/IEC/IEEE 12207:2017 – Systems and Software Engineering – Software Life Cycle Processes,” International Organization for Standardization, inf. téc., 2017.
- [6] J. Mucha, A. Kaufmann y D. Riehle, “A systematic literature review of pre-requirements specification traceability,” *Requirements Engineering*, vol. 29, págs. 119-141, 2 jun. de 2024, ISSN: 0947-3602. DOI: 10.1007/s00766-023-00412-z
- [7] ISO/IEC, “ISO/IEC 25010:2023 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model,” International Organization for Standardization, inf. téc., 2023.
- [8] H. Washizaki, *SWEBOK Guide v4.0*, H. Washizaki, ed. IEEE Computer Society, 2024. dirección: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- [9] H. Washizaki y J. Olszewska, “Guide to the Software Engineering Body of Knowledge v4.0,” vol. 4.0, pág. 413, 2024.
- [10] K. Pohl, *Requirements engineering : fundamentals, principles, and techniques*. Springer, 2025, pág. 814, ISBN: 9783662692042.
- [11] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide) – Seventh Edition*. Project Management Institute, 2021.
- [12] W. Abdeen, X. Chen y M. Unterkalmsteiner, “An approach for performance requirements verification and test environments generation,” *Requirements Engineering*, abr. de 2022, ISSN: 0947-3602. DOI: 10.1007/s00766-022-00379-3
- [13] U.-e. Habiba, M. Haug, J. Bogner y S. Wagner, “How mature is requirements engineering for AI-based systems? A systematic mapping study on practices, challenges, and future research directions,” *Requirements Engineering*, vol. 29, págs. 567-600, 4 dic. de 2024, ISSN: 0947-3602. DOI: 10.1007/s00766-024-00432-3
- [14] B. Li y C. Smidts, “A Zone-Based Model for Analysis of Dependent Failures in Requirements Inspection,” *IEEE Transactions on Software Engineering*, vol. 49, págs. 3581-3598, 6 jun. de 2023, ISSN: 0098-5589. DOI: 10.1109/TSE.2023.3266157
- [15] M. Ruiz, J. Y. Hu y F. Dalpiaz, “Why don’t we trace? A study on the barriers to software traceability in practice,” *Requirements Engineering*, vol. 28, págs. 619-637, 4 dic. de 2023, ISSN: 0947-3602. DOI: 10.1007/s00766-023-00408-9
- [16] P. H. Hein, E. Kames, C. Chen y B. Morkos, “Employing machine learning techniques to assess requirement change volatility,” *Research in Engineering Design*, vol. 32, págs. 245-269, 2 abr. de 2021, ISSN: 0934-9839. DOI: 10.1007/s00163-020-00353-6
- [17] M. Ozkaya, G. Kardas y M. A. Kose, “An Analysis of the Features of Requirements Engineering Tools,” *Systems*, vol. 11, n.º 12, pág. 576, 2023. DOI: 10.3390/systems11120576 dirección: <https://doi.org/10.3390/systems11120576>
- [18] M. A. Nadeem, S. U.-J. Lee y M. U. Younus, “A Comparison of Recent Requirements Gathering and Management Tools in Requirements Engineering for IoT-Enabled Sustainable Cities,” *Sustainability*, vol. 14, n.º 4, pág. 2427, 2022. DOI: 10.3390/su14042427 dirección: <https://doi.org/10.3390/su14042427>
- [19] E. Bjarnason, K. Sharma y P. Runeson, “Software Selection in Large-Scale Software Engineering: A Model and Criteria Based on Interactive Rapid Reviews,” *Empirical Software Engineering*, vol. 28, n.º 2, pág. 51, 2023. DOI: 10.1007/s10664-023-10288-w dirección: <https://doi.org/10.1007/s10664-023-10288-w>
- [20] A. Ferreira, A. R. Da Silva y A. C. R. Paiva, “Towards the Art of Writing Agile Requirements with User Stories, Acceptance Criteria, and Related Constructs,” en *Proceedings of the 17th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE 2022)*, 2022, págs. 477-484. DOI: 10.5220/0011082000003176 dirección: <https://doi.org/10.5220/0011082000003176>

- [21] M. A. Kuhail y S. Lauesen, “User Story Quality in Practice: A Case Study,” *Software*, vol. 1, n.º 3, págs. 223-243, 2022. DOI: 10.3390/software1030010 dirección: <https://doi.org/10.3390/software1030010>
- [22] K. Schwaber y J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*, 2020. dirección: <https://scrumguides.org/>
- [23] G. P. Tomaselli, N. S. Pinto y C. Acuña, “Requirements Management Methods and Practices for Improving the Quality of Agile Software Development Processes: A Review of the Literature,” *Requirements Engineering*, vol. 30, págs. 371-398, 2025. DOI: 10.1007/s00766-025-00448-3 dirección: <https://doi.org/10.1007/s00766-025-00448-3>
- [24] H. M. Abushama, H. A. Alassam y F. A. Elhaj, “The Effect of Test-Driven Development and Behavior-Driven Development on Project Success Factors: A Systematic Literature Review Based Study,” en *2020 International Conference on Computer, Control, Electrical, and Electronics Engineering (ICCCEEE)*, 2021, págs. 1-9. DOI: 10.1109/ICCCEEE49695.2021.9429593 dirección: <https://doi.org/10.1109/ICCCEEE49695.2021.9429593>
- [25] S. Charalampidou, A. Ampatzoglou, E. Karountzos y P. Avgeriou, “Empirical Studies on Software Traceability: A Mapping Study,” *Journal of Software: Evolution and Process*, vol. 33, n.º 2, e2294, 2021. DOI: 10.1002/smr.2294 dirección: <https://doi.org/10.1002/smr.2294>
- [26] S. Loh Mun Keong y Z. Che Embi, “A Systematic Review on Non-Functional Requirements Documentation in Agile Methodology,” *Journal of Informatics and Web Engineering*, vol. 1, n.º 2, págs. 19-29, 2022. DOI: 10.33093/jiwe.2022.1.2.2 dirección: <https://doi.org/10.33093/jiwe.2022.1.2.2>
- [27] IBM, *IBM Engineering Requirements Management DOORS Next*, mar. de 2023. dirección: <https://www.ibm.com/docs/en/engineering-lifecycle-management-suite/doors/9.7.1?topic=overview-doors-nex>
- [28] Jama Software, *Jama Connect Pricing. One Platform. One Simple Licensing Model*. 2026. dirección: <https://www.jamasoftware.com/platform/pricing/>
- [29] ReqView, *ReqView: Requirements Management Tool*, 2026. dirección: <https://www.reqview.com>
- [30] Jama Software, *Only Jama Connect Delivers Live Requirements Traceability*, 2026. dirección: <https://www.jamasoftware.com/solutions/requirements-traceability/>

Anexo A — Lista de verificación de inspección (checklists individuales)

Anexo B — Registro de defectos

Anexo C — Formularios RFC y acta del CCB

Anexo D — Exportación CSV del backlog

Anexo E — Capturas del tablero CASE y del repositorio Git

Anexo F — Referencias IEEE y declaración de uso de Inteligencia Artificial

Referencias

- [1] L. Montgomery, D. Fucci, A. Bouraffa, L. Scholz y W. Maalej, “Empirical research on requirements quality: a systematic mapping study,” *Requirements Engineering*, vol. 27, págs. 183-209, 2 jun. de 2022, ISSN: 0947-3602. DOI: 10.1007/s00766-021-00367-z
- [2] M. A. Umar y K. Lano, “Advances in automated support for requirements engineering: a systematic literature review,” *Requirements Engineering*, vol. 29, págs. 177-207, 2 jun. de 2024, ISSN: 0947-3602. DOI: 10.1007/s00766-023-00411-0
- [3] ISO/IEC/IEEE, “ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Life Cycle Processes – Requirements Engineering,” International Organization for Standardization, inf. téc., 2018.
- [4] ISO/IEC/IEEE, *15288:2023 — System life cycle processes*, 2023.
- [5] ISO/IEC/IEEE, “ISO/IEC/IEEE 12207:2017 – Systems and Software Engineering – Software Life Cycle Processes,” International Organization for Standardization, inf. téc., 2017.
- [6] J. Mucha, A. Kaufmann y D. Riehle, “A systematic literature review of pre-requirements specification traceability,” *Requirements Engineering*, vol. 29, págs. 119-141, 2 jun. de 2024, ISSN: 0947-3602. DOI: 10.1007/s00766-023-00412-z
- [7] ISO/IEC, “ISO/IEC 25010:2023 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model,” International Organization for Standardization, inf. téc., 2023.
- [8] H. Washizaki, *SWEBOK Guide v4.0*, H. Washizaki, ed. IEEE Computer Society, 2024. dirección: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- [9] H. Washizaki y J. Olszewska, “Guide to the Software Engineering Body of Knowledge v4.0,” vol. 4.0, pág. 413, 2024.
- [10] K. Pohl, *Requirements engineering : fundamentals, principles, and techniques*. Springer, 2025, pág. 814, ISBN: 9783662692042.
- [11] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide) – Seventh Edition*. Project Management Institute, 2021.
- [12] W. Abdeen, X. Chen y M. Unterkalmsteiner, “An approach for performance requirements verification and test environments generation,” *Requirements Engineering*, abr. de 2022, ISSN: 0947-3602. DOI: 10.1007/s00766-022-00379-3
- [13] U.-e. Habiba, M. Haug, J. Bogner y S. Wagner, “How mature is requirements engineering for AI-based systems? A systematic mapping study on practices, challenges, and future research directions,” *Requirements Engineering*, vol. 29, págs. 567-600, 4 dic. de 2024, ISSN: 0947-3602. DOI: 10.1007/s00766-024-00432-3
- [14] B. Li y C. Smidts, “A Zone-Based Model for Analysis of Dependent Failures in Requirements Inspection,” *IEEE Transactions on Software Engineering*, vol. 49, págs. 3581-3598, 6 jun. de 2023, ISSN: 0098-5589. DOI: 10.1109/TSE.2023.3266157
- [15] M. Ruiz, J. Y. Hu y F. Dalpiaz, “Why don’t we trace? A study on the barriers to software traceability in practice,” *Requirements Engineering*, vol. 28, págs. 619-637, 4 dic. de 2023, ISSN: 0947-3602. DOI: 10.1007/s00766-023-00408-9
- [16] P. H. Hein, E. Kames, C. Chen y B. Morkos, “Employing machine learning techniques to assess requirement change volatility,” *Research in Engineering Design*, vol. 32, págs. 245-269, 2 abr. de 2021, ISSN: 0934-9839. DOI: 10.1007/s00163-020-00353-6
- [17] M. Ozkaya, G. Kardas y M. A. Kose, “An Analysis of the Features of Requirements Engineering Tools,” *Systems*, vol. 11, n.º 12, pág. 576, 2023. DOI: 10.3390/systems11120576 dirección: <https://doi.org/10.3390/systems11120576>
- [18] M. A. Nadeem, S. U.-J. Lee y M. U. Younus, “A Comparison of Recent Requirements Gathering and Management Tools in Requirements Engineering for IoT-Enabled Sustainable Cities,” *Sustainability*, vol. 14, n.º 4, pág. 2427, 2022. DOI: 10.3390/su14042427 dirección: <https://doi.org/10.3390/su14042427>
- [19] E. Bjarnason, K. Sharma y P. Runeson, “Software Selection in Large-Scale Software Engineering: A Model and Criteria Based on Interactive Rapid Reviews,” *Empirical Software Engineering*, vol. 28, n.º 2, pág. 51, 2023. DOI: 10.1007/s10664-023-10288-w dirección: <https://doi.org/10.1007/s10664-023-10288-w>
- [20] A. Ferreira, A. R. Da Silva y A. C. R. Paiva, “Towards the Art of Writing Agile Requirements with User Stories, Acceptance Criteria, and Related Constructs,” en *Proceedings of the 17th International Conference*

- on *Evaluation of Novel Approaches to Software Engineering (ENASE 2022)*, 2022, págs. 477-484. DOI: 10.5220/0011082000003176 dirección: <https://doi.org/10.5220/0011082000003176>
- [21] M. A. Kuhail y S. Lauesen, “User Story Quality in Practice: A Case Study,” *Software*, vol. 1, n.º 3, págs. 223-243, 2022. DOI: 10.3390/software1030010 dirección: <https://doi.org/10.3390/software1030010>
 - [22] K. Schwaber y J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*, 2020. dirección: <https://scrumguides.org/>
 - [23] G. P. Tomaselli, N. S. Pinto y C. Acuña, “Requirements Management Methods and Practices for Improving the Quality of Agile Software Development Processes: A Review of the Literature,” *Requirements Engineering*, vol. 30, págs. 371-398, 2025. DOI: 10.1007/s00766-025-00448-3 dirección: <https://doi.org/10.1007/s00766-025-00448-3>
 - [24] H. M. Abushama, H. A. Alassam y F. A. Elhaj, “The Effect of Test-Driven Development and Behavior-Driven Development on Project Success Factors: A Systematic Literature Review Based Study,” en *2020 International Conference on Computer, Control, Electrical, and Electronics Engineering (ICCCEEE)*, 2021, págs. 1-9. DOI: 10.1109/ICCCEEE49695.2021.9429593 dirección: <https://doi.org/10.1109/ICCCEEE49695.2021.9429593>
 - [25] S. Charalampidou, A. Ampatzoglou, E. Karountzos y P. Avgeriou, “Empirical Studies on Software Traceability: A Mapping Study,” *Journal of Software: Evolution and Process*, vol. 33, n.º 2, e2294, 2021. DOI: 10.1002/smr.2294 dirección: <https://doi.org/10.1002/smr.2294>
 - [26] S. Loh Mun Keong y Z. Che Embi, “A Systematic Review on Non-Functional Requirements Documentation in Agile Methodology,” *Journal of Informatics and Web Engineering*, vol. 1, n.º 2, págs. 19-29, 2022. DOI: 10.33093/jiwe.2022.1.2.2 dirección: <https://doi.org/10.33093/jiwe.2022.1.2.2>
 - [27] IBM, *IBM Engineering Requirements Management DOORS Next*, mar. de 2023. dirección: <https://www.ibm.com/docs/en/engineering-lifecycle-management-suite/doors/9.7.1?topic=overview-doors-nex>
 - [28] Jama Software, *Jama Connect Pricing. One Platform. One Simple Licensing Model*. 2026. dirección: <https://www.jamasoftware.com/platform/pricing/>
 - [29] ReqView, *ReqView: Requirements Management Tool*, 2026. dirección: <https://www.reqview.com>
 - [30] Jama Software, *Only Jama Connect Delivers Live Requirements Traceability*, 2026. dirección: <https://www.jamasoftware.com/solutions/requirements-traceability/>

Uso de la IA